

GameTools

Reference Folder

A quick-reference and complete reference for the GameTools Kotlin / JVM engine layer — the game-object hierarchy, the stat and buff system, the deterministic tick, spatial indexing, the typed event bus, serializable world-state snapshots, and the UDP game server.

LIBRARY VERSION

3.1.0

BRANCH

master

ISSUED

6 September 2026

AUTHOR

Spartak Singh

How to use this folder

Every chapter is written twice: once for the reader who needs an answer in ten seconds, and once for the reader who needs to understand the machinery.

LAYER ONE — QUICK REFERENCE

Opens every chapter under a coloured band. Tables, cards, diagrams and defaults. Nothing here is prose you have to read in order; it is laid out to be *scanned*. If you already know the concept and only need the signature, the default value, or the order of operations, you never need to turn past this band.

LAYER TWO — IN DETAIL

Follows under a black band, in numbered sub-chapters. Written as a textbook: what the thing is, why it is built the way it is, how it behaves at the edges, and what goes wrong when it is used against the grain. Read it once per concept; return to layer one after.

§ Conventions used throughout

API	A public entry point you are meant to call.	Type.member	A Kotlin declaration, exactly as it is spelled in source.
SAFE	Behaviour you can rely on across versions.	§4.7	Sub-chapter 7 of chapter 4.
CONTRACT	Something <i>you</i> must uphold.	tick	One call to <code>World.tick()</code> — the engine's unit of time.
TRAP	A known way to get this wrong.	unit	One unit of world space. The library fixes no scale.
INTERNAL	Visible, but not part of the contract.		

READING ORDER

First time through: chapter 0, then chapter 1, then whichever concept chapter you need. Chapters 2–8 are independent of one another and can be read in any order. Chapter 9 is a lookup table, not a read. Chapter 10 is worked examples that pull the earlier chapters together.

§ Assembling the physical folder

1 Print US Letter, single-sided, colour. The left margin is widened to 24 mm to clear a three-ring punch.	2 Punch Three-hole punch on the left edge. No text sits within 20 mm of it.	3 Tab Cut the dividers in Appendix E and place one before each chapter opener.	4 Label Trim and slide the spine label in Appendix E into the binder's spine sleeve.
--	---	--	--

ROOM TO ANNOTATE

Prose is set to a narrow measure on purpose. The blank column on the right of every detail page is yours — it is sized for a marginal note in pen next to the paragraph it belongs to.

Contents



Eleven chapters and five appendices. Each chapter carries its own colour, repeated on its divider tab.

00	Quick Reference	7
0.1	The library on one page	
0.2	The type map	
0.3	Sixty seconds to a running world	
0.4	One tick, in one picture	
0.5	Defaults worth memorising	
0.6	House idioms	
0.7	"I want to..." index	
01	Architecture	14
1.1	What GameTools is, and what it is not	
1.2	The layered object model	
1.3	The four composition seams	
1.4	House idioms, and why they exist	
1.5	Modules, artifacts and dependencies	
1.6	The threading model	
1.7	Versioning and the road beyond 3.1	
02	World & Simulation	22
2.1	The World as a container	
2.2	Getting objects in and out	
2.3	The tick lifecycle, step by step	
2.4	Entity identity and the id index	
2.5	The quadtree rebuild contract	
2.6	SimulationLoop: the fixed timestep	
2.7	Determinism, and what breaks it	
03	Game Objects	29
3.1	GameObject — position, activity, the tick hook	
3.2	VisibleObject — what a renderer needs	
3.3	Actor — self-propelled motion	
3.4	The four movement strategies	
3.5	Alive — health, combat, death	
3.6	Player and the ownership invariant	
3.7	Projectiles	
3.8	Extending the hierarchy	

04	Stats, Buffs & Combat	38
4.1	ModularStat and the value formula	
4.2	StatMod: type × stacking	
4.3	CombinedStat: current against a moving ceiling	
4.4	Moddable and the stats map	
4.5	Capabilities	
4.6	Bufs	
4.7	The attack cycle	
4.8	Evasion, damage and attribution	
4.9	Death, respawn and removal	
4.10	The hook catalogue	
05	Spatial Indexing	49
5.1	What a point-region quadtree is	
5.2	The four operations	
5.3	Query semantics: the half-open box	
5.4	Shape, balance and rebuilding	
5.5	Broad phase and narrow phase	
5.6	Limits and traps	
06	Events	54
6.1	Why a bus	
6.2	EventBus semantics	
6.3	The GameEvent catalogue	
6.4	Subscribing safely	
6.5	Patterns	
6.6	Anti-patterns	
07	Network	59
7.1	The transport underneath	
7.2	The handshake	
7.3	Admission, capacity and reconnection	
7.4	Message routing	
7.5	Broadcasting world state	
7.6	Targeted sends and disconnection	
7.7	Keepalives and NAT	
7.8	Threading and lifecycle	
7.9	MouseAction	
7.10	Failure modes	
08	Serialization	67
8.1	The snapshot pattern	
8.2	The drawable family	

8.3	Field-by-field wire reference
8.4	Sub-object trees and visibility
8.5	Identity on the wire
8.6	Compatibility and evolution
8.7	Geometry snapshots

■ **09** **API Reference** **73**

9.1	gaming.gameobjects
9.2	gaming.event
9.3	gaming.simulation
9.4	gaming.spatial
9.5	gaming.networking
9.6	geometry.serializations
9.7	Operators and interop

■ **10** **Recipes** **81**

■ **A–E** **Appendices** **92**

A	Every default in the library
B	Glossary
C	Troubleshooting
D	Build, test and release tasks
E	Printable dividers and spine label

Quick Reference

The whole library at arm's length: what is in it, what it is called, what it does by default, and where the details live.

- | | |
|---|--------------------------------------|
| 0.1 The library on one page | 0.5 Defaults worth memorising |
| 0.2 The type map | 0.6 House idioms |
| 0.3 Sixty seconds to a running world | 0.7 "I want to..." index |
| 0.4 One tick, in one picture | |

QUICK REFERENCE

THIS CHAPTER IS THE AT-A-GLANCE LAYER FOR THE ENTIRE LIBRARY

0.1 The library on one page

WHAT IT IS

A Kotlin/JVM *server-side* engine layer for 2D multiplayer games. It supplies the plumbing — entities, stats, spatial queries, events, snapshots, UDP transport — and leaves the gameplay to you.

WHAT IT IS NOT

Not a client, renderer, physics engine, lobby, matchmaker, database or asset pipeline. It never draws anything and it holds no opinion about your art.

HOW YOU DRIVE IT

You own the clock. Call `World.tick()` once per frame, or hand the world to the opt-in `SimulationLoop`. One world, one thread.

MODULES

MODULE	MAVEN COORDINATES	CONTAINS
gametools-core	io.github.spartanlabsgaming:gametools-core	The object hierarchy, stats, buffs, capabilities, <code>World</code> , <code>EntityId</code> , the event bus, <code>RandomSource</code> , <code>SimulationLoop</code> , <code>Quadtree</code> , geometry snapshots.
gametools-net	io.github.spartanlabsgaming:gametools-net	<code>GameServer</code> and <code>MouseAction</code> . Depends on <code>gametools-core</code> with <code>api</code> , so it drags the core in.
gametools	io.github.spartanlabsgaming:gametools	No source. An umbrella that re-exports every module, for the consumer who wants one dependency line.

The published 3.1.0 release predates the split and ships as the single artifact `io.github.spartanlabsgaming:GameTools:3.1.0`. See §1.5.

RUNTIME DEPENDENCIES

ARTIFACT	VERSION	WHY
io.github.spartanlaboratories:WebTools	2.0.0c	UDP transport — <code>MultiConnectionUDPServer</code> , <code>Connection</code> .
io.github.spartanlaboratories:GeneralTools	2.0.1	<code>Color</code> ; also the source of <code>Point</code> , <code>Dimensions</code> , <code>Square</code> .
org.jetbrains.kotlinx:kotlinx-serialization-json	1.7.3	Every <code>@Serializable</code> snapshot and the JSON wire format.
org.slf4j:slf4j-api	2.0.16	Logging facade only. Bring your own binding.

All four are declared `api`, so they are on your compile classpath too. Kotlin 2.2.0, JVM toolchain 23.

0.2 The type map

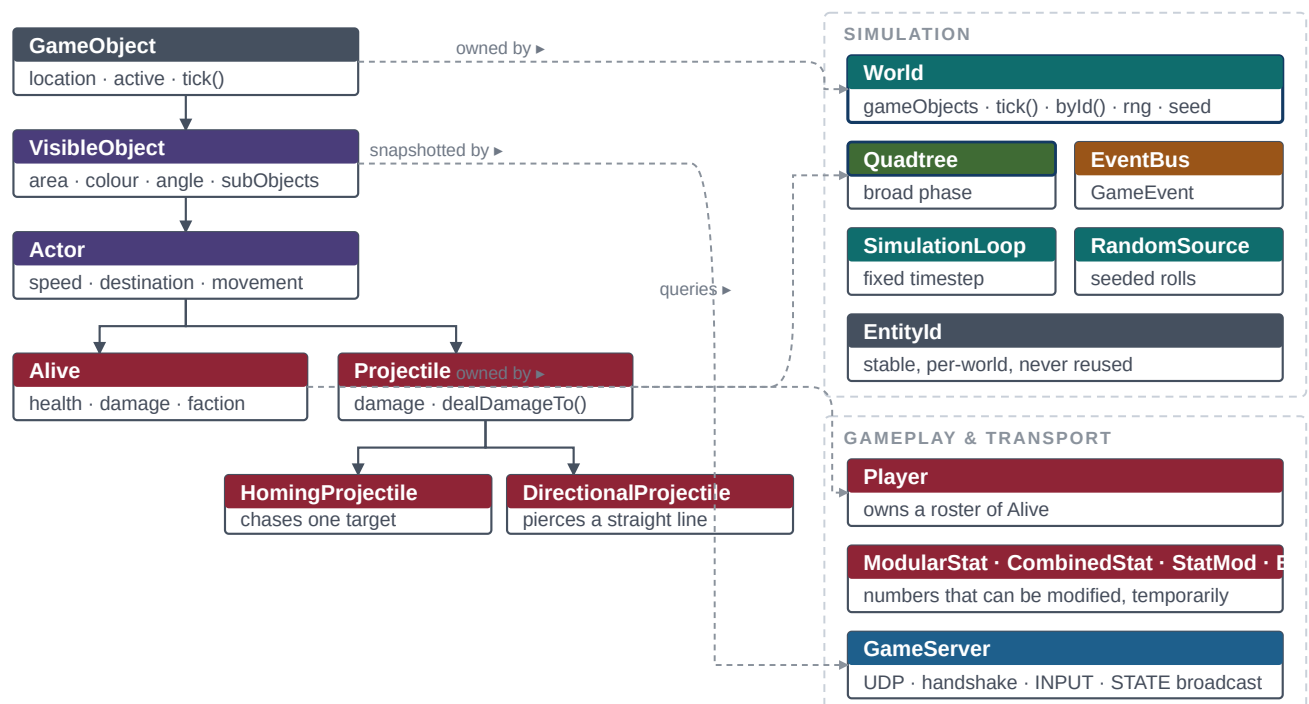


FIGURE 0.1 Solid arrows are inheritance; dashed arrows are collaboration. Each layer of the spine adds exactly one concern. Full treatment in chapter 3.

EVERY PUBLIC TYPE, IN ONE LINE EACH

TYPE	KIND	ONE-LINE PURPOSE	§
GameObject	abstract	Anything the simulation owns: a position, an active flag, buffs, capabilities, a tick hook.	3.1
VisibleObject	open	A game object with something to draw, plus nested sub-objects and box collision.	3.2
Actor	open	A visible object that moves itself under a pluggable strategy.	3.3
Movement	sealed	Targeting · Persistent · Directional · Homing — how an actor picks its step.	3.4
Alive	open	An actor with health, combat stats, a faction, an owner and a death response.	3.5
Player	final	A roster owner. Game-domain, unrelated to a network connection.	3.6
Projectile	abstract	An actor that travels and delivers a damage payload.	3.7
HomingProjectile	final	Chases one Alive , hits once, deactivates.	3.7
DirectionalProjectile	final	Flies a fixed heading for N ticks, damaging each Alive it passes through once.	3.7
World	final	The container and the clock's target: objects, index, quadtree, event bus, RNG.	2.1
EntityId	value	A stable, per-world, never-reused handle for one object.	2.4
SimulationLoop	final	Opt-in daemon thread that ticks a world at a fixed rate.	2.6
LoopSettings	final	Live-tunable tick rate and catch-up cap.	2.6
RandomSource	interface	The only sanctioned source of randomness in the engine.	2.7
SeededRandom	final	The default RandomSource , seeded.	2.7
ModularStat	final	A Double reshaped by named modifiers; behaves like a number.	4.1
StatMod	data	One named adjustment: additive or multiplicative, with a stacking rule.	4.2
CombinedStat	data	A current/max pair that rescales when the ceiling moves.	4.3
Moddable	interface	What both stat kinds have in common, so a buff need not care which it holds.	4.4
Capability	interface	A named ability that a buff can suppress. CoreCapability supplies MOVE and ATTACK.	4.5
Buff	open	A timed bundle of stat mods and capability suppressions, with three hooks.	4.6
Quadtree<N, E>	final	Point-region spatial index for rectangular range queries.	5.1
EventBus	final	Synchronous, in-order, fault-isolated publish/subscribe.	6.2
GameEvent	sealed	Eight facts the simulation reports about itself.	6.3
GameServer	final	UDP server: handshake, admission, input decoding, JSON broadcast.	7.1
MouseAction	data	A typed, serializable mouse event in window pixels.	7.9
DrawableSnapshot	sealed	The polymorphic wire type for one drawable object.	8.2

0.3 Sixty seconds to a running world

```
// build.gradle.kts
dependencies {
    implementation(
        "io.github.spartanlabsgaming:GameTools:3.1.0")
    runtimeOnly(
        "ch.qos.logback:logback-classic:1.5.18")
}
```

```
import com.spartanlabs.gaming.gameobjects.*
import com.spartanlabs.geometry.Dimensions
import com.spartanlabs.geometry.Point

val world = World(seed = 42L)
val player = Player("Spartak")

val hero = Alive(
    location    = Point(x = 100.0, y = 100.0),
    dimensions  = Dimensions(40.0, 40.0),
    maxHealth   = 250.0)

player.own(hero)
world.add(hero)                // not gameObjects +=

hero.destination = Point(x = 400.0, y = 250.0)

while (running) {
    world.tick()                // you own the clock
}
```

THE FIVE CALLS THAT MATTER

- `World()` — or `World(seed)` to make the run reproducible.
- `world.add(obj)` — numbers it, indexes it, wires `Alive.world`.
- `actor.destination = p` — also re-aims `angle`.
- `alive.issueAttack(other)` — starts the chase-and-swing loop.
- `world.tick()` — advances everything by one frame.

TRAP

`world.gameObjects += hero` compiles and mostly works, but it skips `add()`: the object is not resolvable by id until the next tick, and an `Alive` is left with `world == null`, so a `REMOVAL` death has nowhere to queue itself and the corpse stays in play. Prefer `add()`. See §2.2.

THEN, OPTIONALLY

Hand the world to a `SimulationLoop` (§2.6) instead of writing the `while` loop, and stand up a `GameServer` (§7) to broadcast `DrawableSnapshot`s to clients once per tick.

0.4 One tick, in one picture

WORLD.TICK() — ONCE PER FRAME, ON YOUR THREAD

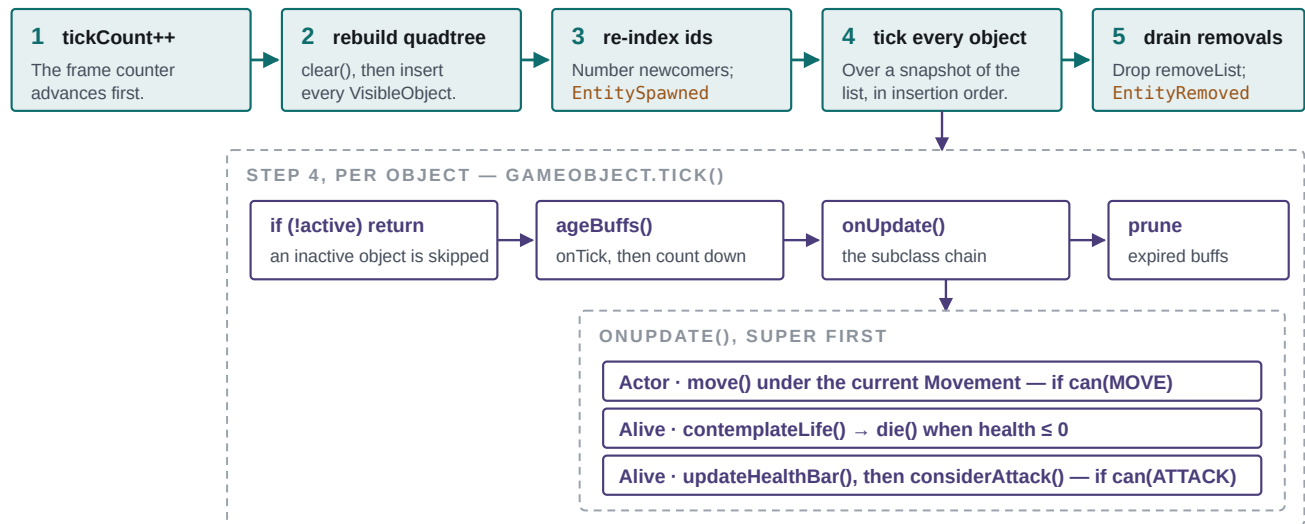


FIGURE 0.2 The five phases of `World.tick()`, and what happens inside one object's turn. Events are delivered the instant they are published, not batched. Full treatment in §2.3.

0.5 Defaults worth memorising

GEOMETRY AND RENDERING

PROPERTY	DEFAULT
<code>GameObject.location</code>	<code>Point()</code> — origin
<code>GameObject.active</code>	<code>true</code>
<code>GameObject.entityId</code>	<code>UNASSIGNED (0)</code>
<code>VisibleObject.dimensions</code>	<code>25.0 × 25.0</code>
<code>VisibleObject.color</code>	<code>Color(255,255,255,255)</code>
<code>VisibleObject.texture</code>	<code>"default.png"</code>
<code>VisibleObject.angle</code>	<code>0</code> (degrees CCW)
<code>VisibleObject.turns</code>	<code>false</code>
<code>VisibleObject.visible</code>	<code>true</code>
<code>Actor.speed</code>	<code>10.0</code> units / tick
<code>Actor.movement</code>	<code>Movement.Targeting</code>
<code>Actor.destination</code>	a copy of location

COMBAT, SIMULATION AND WIRE

PROPERTY	DEFAULT
<code>Alive.damage</code>	<code>10.0</code>
<code>Alive.attackTime</code>	<code>1.7</code>
<code>Alive.attackSpeed</code>	<code>100.0</code>
<code>Alive.attackRange</code>	<code>750.0</code> units
<code>Alive.evasion</code>	<code>0.0</code> (never dodges)
<code>Alive.faction</code>	<code>"neutral"</code>
<code>Alive.deathResponse</code>	<code>REMOVAL</code>
<code>health bar height</code>	<code>20 %</code> of actor height
<code>World.seed</code>	<code>Random.nextLong()</code>
<code>LoopSettings.tickRateHz</code>	<code>20.0</code>
<code>LoopSettings.maxCatchUpTicks</code>	<code>5</code>
<code>StatMod</code> type / stacking	<code>MULTIPLICATIVE / ADDITIVE</code>

THE DEFAULT SWING IS SLOW — DELIBERATELY CHECK IT

Swing progress accrues at `attackSpeed / 10000` per tick, and a swing lands when it reaches `attackTime`. With the stock numbers that is $100/10000 = 0.01$ per tick against a threshold of `1.7` — **170 ticks per swing**, or 8.5 seconds at the default 20 Hz. Almost every game will want to raise `attackSpeed` or lower `attackTime`. See §4.7.

0.6 House idioms

RESULT, NOT EXCEPTIONS

Operational failure comes back as `Result.failure`. Nothing in the library throws for an expected condition. Constructor `require` checks are the exception — a bad constructor argument is a programming error, not an operational one.

```
actor.move().onFailure { ... }
```

SNAPSHOT, NOT THE LIVE OBJECT

Every networkable type has an immutable `@Serializable` twin whose name is the type plus `Snapshot`, built through an `infix` companion function called `from`.

`AliveSnapshot` from `hero`

HOOKS, NOT CALLBACKS

Extension points are `protected open fun on...()` members you override in a subclass, and they do nothing by default. Both sides of an interaction get one: `onHitting` fires on the attacker, `onHitBy` on the target.

SUPER FIRST

An overridden `onUpdate()` calls `super.onUpdate()` before its own work, so an ancestor's per-frame behaviour still happens. Break this and an `Alive` subclass silently stops moving.

STATS BEHAVE LIKE NUMBERS

`ModularStat` and `CombinedStat` define the arithmetic and comparison operators in both directions, so `if (distance > attackRange)` compiles without unwrapping `.value`.

STRUCTURED LOGGING, FACADE ONLY

Every layer logs lifecycle and failure through `slf4j` with `{}` placeholders, and binds to no implementation. Add `Logback` yourself to see any of it.

0.7 “I want to...” index

I WANT TO...	GO TO	I WANT TO...	GO TO
...get a unit moving	\$3.3, \$3.4	...build a kill feed or a score	\$6.3, \$10.5
...make a unit chase and attack another	\$4.7	...react to a spawn without touching World	\$6.3
...break off an attack mid-fight	\$4.7	...accept players over the network	\$7.2, \$10.1
...apply a haste, a slow or a stun	\$4.6, \$10.3	...decode a client's click into an order	\$7.4, \$10.6
...understand why a buff's numbers vanished	\$4.6	...send world state to clients	\$7.5, \$8.2
...make a unit respawn instead of despawn	\$4.9	...address one object from a client	\$2.4, \$8.5
...fire a homing missile	\$3.7, \$10.4	...hide an object from clients but keep it simulating	\$3.2, \$8.4
...fire a piercing beam	\$3.7, \$10.4	...reproduce a bug exactly	\$2.7, \$10.7
...find everything near a point	\$5.2, \$5.5	...run the simulation on its own thread	\$2.6, \$10.9
...write my own collision test	\$5.5	...add my own entity type	\$3.8, \$10.8

Architecture

How the library is put together, why it is shaped that way, and the handful of conventions that make the rest of it predictable.

- 1.1 What GameTools is, and what it is not
- 1.2 The layered object model
- 1.3 The four composition seams
- 1.4 House idioms, and why they exist

- 1.5 Modules, artifacts and dependencies
- 1.6 The threading model
- 1.7 Versioning and the road beyond 3.1

QUICK REFERENCE

LAYERING · SEAMS · IDIOMS · MODULES

THE LAYERS, TOP TO BOTTOM

LAYER	KIND	ADDS EXACTLY
GameObject	abstract	A position, an <code>active</code> flag, buffs, capabilities, and the <code>tick()</code> lifecycle every entity shares.
VisibleObject	open	Rendering state — area, colour, texture, angle, visibility — nested <code>subObjects</code> , and axis-aligned box collision.
Actor	open	Self-propelled motion through a pluggable <code>Movement</code> strategy and a <code>speed</code> stat.
Alive	open	Combat: health, damage, swing timing, range, evasion, faction, <code>Player</code> ownership, death.
Projectile	abstract	A travelling damage payload; two concrete flavours ship with the library.
Player	final	A roster of <code>Alive</code> actors under one controller, kept in a two-way invariant.
World	final	Ownership of every object, the id index, the quadtree, the event bus, the RNG, the tick order.
EventBus	final	Synchronous, in-order, fault-isolated delivery of <code>GameEvent</code> s.
SimulationLoop	final	An <i>optional</i> fixed-timestep driver. Nothing in the engine depends on it.
Quadtree	final	A generic point-region index; the broad phase for every proximity question.
GameServer	final	UDP transport, handshake, admission, input decoding, JSON world-state broadcast.

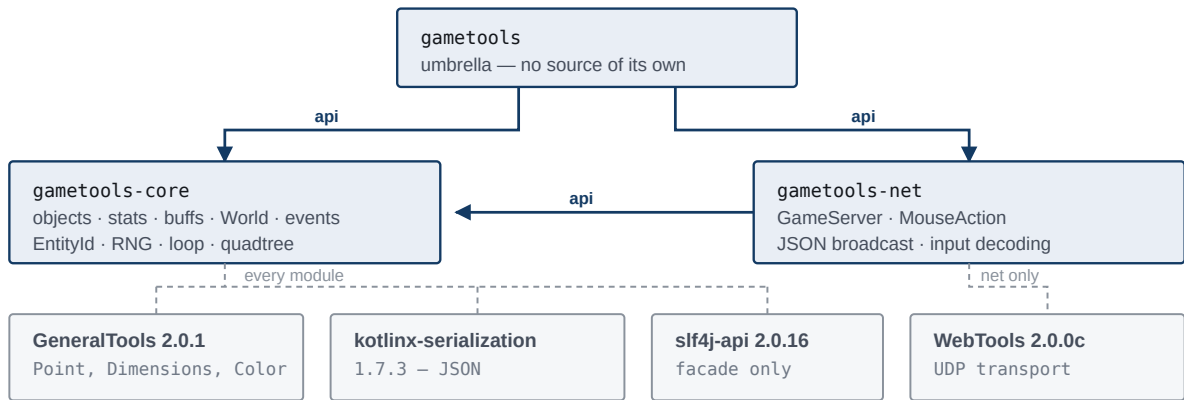


FIGURE 1.1 Every dependency edge is declared **api**, not **implementation**: a consumer of any module compiles against the whole transitive set.

RULE 1 · ONE CONCERN PER LAYER

A new layer earns its place by adding exactly one thing. If a feature does not fit that test, it becomes a seam instead.

RULE 2 · RESULT, NEVER A THROWN SURPRISE

Expected failure is a value. Exceptions are reserved for programming errors caught at construction.

RULE 3 · THE ENGINE NEVER OWNS THE CLOCK

`World.tick()` is the primitive. The loop that calls it is yours, or an opt-in convenience.

RULE 4 · WIRE TYPES ARE SEPARATE TYPES

Nothing live is serialized. Every payload is an immutable snapshot taken at a known instant.

RULE 5 · RANDOMNESS HAS ONE DOOR

Every roll goes through `RandomSource`. A seed plus an input sequence reproduces a run.

RULE 6 · LOG THE FACADE, BIND NOTHING

Modules log through `slf4j` and ship no binding, so a consumer keeps control of its own logging stack.

WHERE A CONCERN LIVES

IF YOU NEED TO CHANGE...	...THE SEAM TO REACH FOR	§
how an entity moves	assign a different <code>Movement</code> to <code>Actor.movement</code>	3.4
a number, temporarily	apply a <code>StatMod</code> , or a <code>Buff</code> carrying one	4.2
a number, permanently	assign <code>ModularStat.base</code>	4.1
what an entity may do right now	suppress a <code>Capability</code> through a <code>Buff</code>	4.5
behaviour at a moment in combat	override a <code>protected open fun on...</code> hook	4.10
what happens elsewhere when X occurs	subscribe to the <code>GameEvent</code> on <code>World.events</code>	6.3
what a client is told	filter the list you hand to <code>GameServer.broadcast</code>	7.5
an entirely new kind of entity	subclass at the shallowest layer that fits	3.8

IN DETAIL

§1.1 – §1.7

1.1 What GameTools is, and what it is not

GameTools is a Kotlin/JVM library that supplies the reusable machinery underneath a small-to-medium 2D multiplayer game: a hierarchy of simulated objects, a system of numbers that other numbers can modify, a spatial index, a stream of typed events, a way of turning live state into bytes, and a UDP server that ships those bytes to players. It was extracted from Spartan Laboratories' own game projects so that the plumbing could be improved once and reused everywhere, rather than re-implemented per title.

The boundary is drawn deliberately and it is worth stating in the negative, because most confusion about the library comes from expecting something on the far side of it. GameTools contains no client and no renderer: it computes where things are and what colour they are, and stops. It contains no physics beyond axis-aligned overlap tests — there is no mass, no restitution, no continuous collision detection. It contains no lobby, no matchmaking and no concept of more than one concurrent game; the design assumes one `World` per server process, and running several matches means running several processes. It contains no persistence, no accounts, and no database adapter. And it does not run itself: there is no hidden thread starting on your behalf when you construct a `World`.

What you get in return for those omissions is a library you can hold entirely in your head. The whole of it is roughly three thousand lines of Kotlin across two modules, every public declaration carries KDoc, and the tick order is documented to the step. When a simulation misbehaves, the answer is somewhere in a code path you can read in an afternoon.

AUTHORITATIVE BY CONSTRUCTION

Because the library models the server side only, everything it computes is authoritative by default. Clients receive snapshots and send input; they never tell the simulation what happened. That asymmetry is baked into the type system — `MouseAction` travels inbound, `DrawableSnapshot` travels outbound, and there is no inbound state type at all.

1.2 The layered object model

The central design decision is that an entity is not one class with forty properties but a chain of four, each of which adds one concern and knows nothing about the ones below it. A `GameObject` knows where it is and whether it is participating. A `VisibleObject` additionally knows how to be drawn. An `Actor` additionally knows how to move. An `Alive` additionally knows how to fight and how to die. Each layer's per-frame work is added by overriding `onUpdate()` and calling `super.onUpdate()` first, so a single call to `tick()`

on an `Alive` runs the buff machinery, then the movement step, then the death check, the health-bar refresh and the attack step, in that order and with no coordinating code anywhere.

The practical benefit is that you subclass at the shallowest layer that fits. A decorative fog bank that drifts across the map is an `Actor`, not an `Alive` with its combat fields ignored; a trigger volume that has no appearance at all is a bare `GameObject`. Choosing the right depth costs nothing at the keyboard and saves the per-tick cost of machinery the object will never use — an `Alive` runs a death check, a health-bar update and an attack-state evaluation on every single tick whether or not it will ever fight.

The cost of the design is the one every deep hierarchy pays: behaviour is inherited, not composed, so an entity cannot be an `Alive` that does not move, and the layer boundaries are fixed at compile time. The library softens this with the seams described next — in particular capabilities, which let an object that structurally *can* move be temporarily forbidden from doing so without changing its type.

1.3 The four composition seams

Where inheritance would be too coarse, GameTools uses one of four seams. Learning to recognise which seam a problem calls for is most of what it takes to use the library well.

1.3.1 Strategy: Movement

An `Actor` does not implement movement; it delegates to a `Movement` object held in a mutable property. The four supplied strategies are Kotlin `data objects` and a single `data class`, all stateless with respect to the actor, so assigning one is free and switching mid-flight is legitimate. A projectile changes its own strategy in its constructor; nothing prevents your code from doing the same when a unit is knocked back, feared or ordered to patrol.

1.3.2 Modifier: StatMod on a Moddable

Every number that gameplay might want to alter is a `ModularStat` rather than a `Double`. A `ModularStat` keeps a `base` and a list of named modifiers, and recomputes its effective `value` whenever that list changes. Because the type also defines the arithmetic and comparison operators against `Double` and against itself, in both directions, a stat is a drop-in replacement for a plain number at almost every call site. This is what allows a haste effect to be layered on and peeled off cleanly rather than being folded destructively into the underlying value.

1.3.3 Gate: Capability

A capability is a named thing a type can do. `Actor` declares `MOVE`; `Alive` adds `ATTACK`. Each layer gates its own per-tick work behind `can(capability)`, which is true when the object's type declares the capability *and* no active buff is suppressing it. This is how a root, a disarm and a stun are expressed without a single new field on `Actor`: a root suppresses `MOVE`, a disarm suppresses `ATTACK`, a stun suppresses both. Your own subclasses declare their own capabilities and gate on them the same way.

1.3.4 Notification: hooks and the event bus

There are two notification mechanisms and they answer different questions. A *hook* is a `protected open fun` on the object itself — `onHitBy`, `onDeath`, `onAttackEnded` — and it answers “what does *this kind of entity* do when this happens to it?” An *event* is a `GameEvent` published on `World.events`, and it answers “what does the *rest of the system* do when this happens anywhere?” Scoring, kill feeds, network deltas and aggro tables belong on the bus; a boss's enrage-at-half-health belongs in a hook.

1.4 House idioms, and why they exist

The codebase follows a written manifesto, and the conventions are consistent enough that they function as a reading aid. Five of them affect you as a consumer.

Failure is a value. Operations that can fail for an operational reason return `Result`. `Actor.move()`, `SimulationLoop.start()`, `GameServer.push()` and `GameServer.broadcast()` all report this way, and the calling code composes with `onFailure`, `map` and `fold` rather than `try/catch`. Exceptions are used in one place only: `require` checks in constructors, where a bad argument is a bug in your code and not a runtime condition to recover from. `CombinedStat` rejects a non-positive maximum, `DirectionalProjectile` rejects a non-positive lifetime and `LoopSettings` rejects a non-positive tick rate, all by throwing at construction.

Wire state is a separate, immutable type. No live object is `@Serializable`. Instead, each networkable type has a companion data class whose name is the type plus `Snapshot`, created through an `infix` companion function named `from`: `AliveSnapshot from hero`. The pattern makes the instant of capture explicit and keeps mutable simulation state from being handed to a serializer mid-tick.

Extension is by overriding, not by registering. The library exposes no listener registry on individual objects. Behaviour is added by subclassing and overriding a hook, which keeps the per-tick cost at zero for the common case where nothing is overridden.

Logging is structured and unbound. Every layer holds an slf4j `Logger` and logs lifecycle transitions at `debug`, notable state at `info`, and recoverable problems at `warn`, always with `{}` placeholders rather than string concatenation. The library binds no implementation, so until you put Logback or another binding on the classpath you will see nothing at all — which is the single most common “why is it silent?” question. The world's seed is logged at `info` on construction specifically so a failing run can be reproduced.

Nullability carries meaning. Where a reference can legitimately be absent, it is nullable and the null case is documented as a signal rather than an error: `World.byId` returns `null` for an object that has died or been removed, which is exactly the answer a command handler needs when a client names a stale target.

1.5 Modules, artifacts and dependencies

The source tree is split into three Gradle modules. `gametools-core` holds everything that simulates: the object hierarchy, stats, buffs, capabilities, `World`, `EntityId`, the event bus, the random source, the simulation loop, the quadtree and the geometry snapshots. `gametools-net` holds the two types that touch a socket, `GameServer` and `MouseAction`, and depends on the core. `gametools` is an umbrella with no source of its own that re-exports both, so a consumer who does not care about the split can depend on one coordinate and get everything, exactly as the pre-split artifact behaved.

Every internal dependency edge is declared with Gradle's `api` configuration rather than `implementation`. That is a deliberate choice for a library whose types appear in its own public signatures: `GameServer.broadcast` takes `DrawableSnapshot`, which lives in the core, so the core must be on the consumer's compile classpath. The same reasoning applies to the four external dependencies — WebTools, GeneralTools, kotlin-serialization and slf4j are all `api`, and all four are therefore visible to your build.

COORDINATES IN FLIGHT

The module split is on `master` but has not yet been released. The published 3.1.0 artifact is the single, pre-split `io.github.spartanlabsgaming:GameTools:3.1.0`; the per-module coordinates `gametools-core`, `gametools-net` and `gametools` are declared in the build files and will appear on Maven Central with the next release. Until then, depend on `GameTools`.

Two of the four external dependencies are Spartan Laboratories' own. WebTools supplies the UDP transport that `GameServer` extends — `MultiConnectionUDPServer` and `Connection` — and its wire behaviour is part of GameTools' observable protocol, so its version matters more than a

typical transitive dependency's (see §7.1). GeneralTools supplies `Color` and the geometry primitives `Point`, `Dimensions`, `Square` and `TwoDoubles` on which the entire object model is built. Neither is optional.

1.6 The threading model

The model is deliberately austere: **one world, one thread**. Nothing in `gametools-core` synchronises anything. `World.tick()`, the objects it ticks, the quadtree it rebuilds and the event bus it publishes on are all designed to be touched by exactly one thread, and if two threads mutate a world concurrently the failure mode is corruption rather than an exception.

Two places deviate, and both do so explicitly. `SimulationLoop` moves the tick to a daemon thread of its own — but it moves *all* of it, including the `onTick` callback, so the rule becomes “one world, one thread, and that thread is the loop's.” Once a loop is running you must stop mutating the world from anywhere else. `GameServer`, by contrast, is genuinely concurrent: WebTools runs a handshake thread and a listener thread per player, so `onPlayerInput` and `onPlayerMessage` are invoked off the simulation thread and may run concurrently for different players. The server's own player table is a `ConcurrentHashMap` for that reason. Your callbacks must not touch the world directly; the correct shape is to queue the decoded input and drain the queue at the top of your tick (§10.6).

THE MOST EXPENSIVE MISTAKE IN THE LIBRARY

Calling `world.add(...)` or moving an actor from inside `onPlayerInput` works in a single-player test and corrupts the world under load. Queue the input; apply it on the tick thread.

1.7 Versioning and the road beyond 3.1

Releases follow a *Major.Feature.MinorChange* scheme, with a trailing letter marking a bug-fix release, and are tagged `vX.Y.Z`. The changelog is kept in the Keep a Changelog format. The 3.0.0 major bump was a coordinate change plus a breaking wire-protocol change inherited from WebTools 2.0.0c; 3.1.0 added stable entity ids, the event bus, the seeded tick, `cancelAttack` and the simulation loop.

The direction beyond 3.1 is recorded in `docs/framework-vision-and-roadmap.md` and is worth knowing when you decide how much to build on top of the current surface. The target is a scalable authoritative server: per-tick delta snapshots keyed by `EntityId`, per-player interest filtering driven by vision and fog of war, a pluggable `SnapshotCodec` with a compact binary implementation alongside today's JSON, a tiled map model with terrain and pathfinding, richer combat with damage types and

threat, abilities and items, and further module splits along those lines. The pieces added in 3.1 — stable ids, the event stream and the deterministic tick — exist because every one of those features needs them first.

World & Simulation

The container that owns everything, the frame that advances it, and the guarantees that make two runs of the same input come out the same.

- 2.1 The World as a container
- 2.2 Getting objects in and out
- 2.3 The tick lifecycle, step by step
- 2.4 Entity identity and the id index

- 2.5 The quadtree rebuild contract
- 2.6 SimulationLoop: the fixed timestep
- 2.7 Determinism, and what breaks it

QUICK REFERENCE

WORLD · TICK ORDER · SIMULATIONLOOP · DETERMINISM

WORLD — THE WHOLE SURFACE

MEMBER	MEANING
<code>World(seed: Long)</code>	Constructor. Defaults to <code>Random.nextLong()</code> ; the seed is logged at <code>info</code> .
<code>seed: Long</code>	The value <code>rng</code> was built from. Pin it to replay a run.
<code>rng: RandomSource</code>	The only sanctioned randomness for objects this world owns.
<code>tickCount: Long</code>	Frames elapsed. <code>0</code> before the first tick. Read-only.
<code>gameObjects: ArrayList</code>	Every owned object, in insertion order. Public and mutable.
<code>removeList: ArrayList</code>	Objects to drop at the end of the current tick.
<code>quadtree: Quadtree</code>	Rebuilt from scratch at the start of every tick.
<code>events: EventBus</code>	Where every <code>GameEvent</code> is published.
<code>add(gameObject)</code>	Add, number, index, and wire <code>Alive.world</code> . The preferred entry point.
<code>byId(id): GameObject?</code>	Resolve an <code>EntityId</code> , or <code>null</code> once the object is gone.
<code>tick()</code>	Advance one frame. Five phases, in the fixed order below.

THE ONE-LINE SUMMARY

A `World` is a list, three indexes and a clock input. It owns objects; it does not schedule itself.

EVERYTHING IS PUBLIC

`gameObjects` and `removeList` are exposed `ArrayList`s. That is convenience, not an invitation: mutate them and you opt out of numbering, indexing and `Alive.world` wiring.

NO BOUNDS

A world has no size, no walls and no coordinate limits. Keeping objects on the map is your job.

THE FIVE PHASES OF WORLD.TICK()

#	PHASE	WHAT HAPPENS	EVENTS PUBLISHED
1	tickCount++	The frame counter advances before anything observes it.	—
2	rebuildQuadtree()	<code>clear()</code> , then insert every <code>VisibleObject</code> in <code>gameObjects</code> at its current position.	—
3	reindexEntities()	Clear the id map, re-enrol every object and its sub-object tree, number newcomers, forget departed ids.	EntitySpawned
4	tick every object	Over a copy of the list taken before the pass, in insertion order. Additions made during the pass wait for the next frame.	AttackIssued · AttackLanded · DamageDealt · EntityDied · AttackCancelled · AttackEnded
5	drain removeList	Remove the queued objects (de-duplicated), drop their ids, clear the list.	EntityRemoved

Events are delivered synchronously at the moment of publication, not batched at the end of the tick.

SIMULATIONLOOP

MEMBER	MEANING
<code>SimulationLoop(world, settings, onTick)</code>	Constructs, but does not start. <code>onTick</code> receives the new <code>tickCount</code> .
<code>start(): Result</code>	Spawns the daemon thread <code>gametools-sim-loop</code> . Fails if already running.
<code>stop(): Result</code>	Signals, unparks and joins, waiting up to 1 s. Idempotent.
<code>isRunning: Boolean</code>	Whether <code>start</code> has been called and <code>stop</code> has not.
<code>advance(realElapsedNanos)</code>	Step the timestep yourself, from your own loop. Not safe alongside a running thread.
<code>settings</code>	The live-tunable <code>LoopSettings</code> .

LOOPSETTINGS

KNOB	DEFAULT	RULE
<code>tickRateHz</code>	20.0	Must be positive and finite; a bad value throws. Volatile — change it from any thread.
<code>maxCatchUpTicks</code>	5	Coerced to at least 1. Owed time past the cap is discarded.
<i>The loop parks half a tick at a time, so a rate change takes effect within one frame.</i>		

DETERMINISM CHECKLIST

A run reproduces exactly when all five hold: 1. the world was built from a fixed `seed`; 2. every random choice went through `World.rng`; 3. objects were added in the same order; 4. the same external calls were made on the same tick numbers; 5. nothing consulted wall-clock time, a hash-ordered collection, or another thread.

THE FOUR DETERMINISM BREAKERS

`Math.random()` or `Random.Default` anywhere in your subclasses · iterating a `HashSet` of objects and acting on the order · applying input the moment a datagram arrives instead of on a tick boundary · driving the world from more than one thread.

2.1 The World as a container

A `World` is the boundary of one simulation. It holds a flat `ArrayList` of every `GameObject` that exists, three derived structures over that list — a quadtree of the visible ones, a hash index from `EntityId` to object, and a set of ids that have already been announced — plus an event bus and a seeded random source. It holds no map, no terrain, no bounds and no notion of teams; those are your game's concern, and the library's roadmap places them in a future `gametools-world` module.

Construction takes a single optional argument, the seed. Passing one makes the run reproducible; omitting it draws a fresh `Random.nextLong()`, which is then written to the log at `info` level precisely so that an interesting failure can be replayed by pinning the value from the log line. The seed is exposed as `World.seed` and the generator built from it as `World.rng`. The property is named `rng` rather than `random` for an unglamorous but real reason: inside a caller's `apply { }` block, a member called `random` would shadow the caller's own `random`.

The list is public and mutable, which is a pragmatic decision rather than an oversight — filtering, counting and inspecting the world is common enough that wrapping it in accessors would be noise. The consequences of mutating it directly are described in the next section.

2.2 Getting objects in and out

There are two ways in and one way out. The way in that you should use is `World.add(gameObject)`, which does four things: appends to `gameObjects`; assigns an `EntityId` if the object does not have one and indexes it, recursing into its `subObjects` tree; sets `Alive.world` when the object is an `Alive`; and publishes `EntitySpawned` immediately.

The other way in is `world.gameObjects += obj`. This is legal and the object will be simulated from the next tick, because phase 3 of the tick re-indexes the whole list and will discover it. But between the addition and that tick the object is invisible to `byId`, and — the part that actually bites — an `Alive` added this way has `world == null`. An `Alive` with no world and the default `REMOVAL` death response has nowhere to queue itself when it dies: the library logs a warning and the corpse stays in play at zero health forever. If you must add directly, set `alive.world = this` yourself.

The way out is `world.removeList += obj`. Nothing is removed the moment you ask; the object is queued and dropped in phase 5, after every object has ticked. That indirection is what makes it safe for an object to remove itself

during its own update — which is exactly what a dying `Alive` does. Removal de-duplicates, so queueing the same object twice in one tick removes it once and publishes one `EntityRemoved`.

REMOVAL IS NOT DEACTIVATION

Setting `active = false` keeps an object in the world and in the quadtree; it simply stops being ticked, and — for a `VisibleObject` — stops being sent to clients, because `active` drives `visible`. Queueing into `removeList` takes it out of the world entirely and releases its id. Projectiles deactivate when spent; they do not remove themselves, so a long-running world will accumulate inert projectiles unless you sweep them.

2.3 The tick lifecycle, step by step

One call to `World.tick()` is one frame of simulated time. The order of its five phases is part of the contract, not an implementation detail, and understanding it explains most of the library's apparent surprises.

Phase 1 — the counter. `tickCount` is incremented first, so any code that observes it during the tick sees the number of the frame being computed, not the one just finished.

Phase 2 — the quadtree. The tree is cleared and every `VisibleObject` in the list is re-inserted at its current position. This happens *before* any object moves, so throughout the tick every object querying the tree sees the same, consistent snapshot of where everything was at the start of the frame. The cost of that consistency is one frame of staleness in collision tests, which for a 20 Hz simulation is the right trade.

Phase 3 — identity. The id map is cleared and rebuilt by walking the list in order, assigning ids to anything unnumbered and recursing into sub-object trees so that health bars and nameplates are addressable too. Any top-level object seen for the first time triggers an `EntitySpawned`; ids belonging to objects that have left are forgotten, so an instance that is removed and later re-added is announced again.

Phase 4 — the objects. A copy of the list is taken and every object in it is ticked, in insertion order. Iterating a copy is what allows an object to append to `gameObjects` or `removeList` during its own update without disturbing the pass; the price is that an object spawned mid-tick is neither numbered nor ticked until the next frame. Ordering is insertion order, deterministically, which is why the order in which you add objects is part of the determinism contract.

Phase 5 — removals. Whatever accumulated in `removeList` is de-duplicated, dropped from the list and from the id index, and announced with one `EntityRemoved` each. The list is then cleared.

Inside phase 4, each object runs its own three-step sequence: every active buff has its `onTick` fired and its remaining duration decremented; `onUpdate()` runs the subclass chain; and any buff that reached zero this tick is removed, undoing its stat mods and releasing its capability suppressions. The consequence worth memorising is that a buff applied with a duration of n is in force for exactly n updates.

2.4 Entity identity and the id index

Before version 3.1 the only way for a client to refer to an object was its position in the broadcast list, which changes whenever anything spawns or dies. `EntityId` fixes that. It is a `@JvmInline value class` over a `Long`, so it costs nothing at runtime, and it prints as `#7`.

A world numbers each object it takes ownership of, in acquisition order, starting from `1`. The number is assigned exactly once and never changes — re-adding the same instance, even to a different world, keeps the id it was first given — and it is never reused for a different object. `EntityId.UNASSIGNED`, whose raw value is `0`, marks an object no world has numbered yet. That zero doubles as the decode default on the wire, so “this object is not addressable” and “this payload predates stable ids” collapse into one case a client can handle uniformly.

Resolution goes through `World.byId(id)`, which returns the object or `null`. The null is the useful part: a client that clicks a unit and sends a command three frames later may well be naming something that has since died, and `byId` returning `null` is precisely the signal the command handler needs. Objects added through `add()` resolve immediately; objects appended directly to `gameObjects` resolve from the next tick.

IDS ARE PER WORLD AND PER PROCESS

Two different worlds hand out the same raw values to different objects, so an id is meaningful only alongside the world that issued it. Treat the number as opaque: compare it, address by it, but do not parse it, derive anything from its magnitude, or persist it across a restart.

2.5 The quadtree rebuild contract

The world's `quadtree` is not a cache that is incrementally maintained; it is discarded and rebuilt from scratch at the top of every tick. This is the single most important thing to know about using it, because it fixes what the tree contains at any moment:

- During phase 4 — that is, inside any object's `onUpdate()` — the tree reflects every object's position *as of the start of this tick*, including objects that have already moved this frame.
- Between ticks, the tree reflects positions as of the start of the *last* tick.
- Only `VisibleObjects` are in it. A bare `GameObject` is never indexed, and neither are `subObjects` — the tree holds top-level objects only.
- Inactive and invisible objects *are* in it. The rebuild filters on type, not on state, so a deactivated projectile still appears in query results until it is removed from the world. Filter for `active` yourself if that matters.

If you drive collision from your own code outside the tick, you own the freshness of the tree: clear it and re-insert, exactly as the world does. The projectile classes document this requirement explicitly because they query a tree passed in by the caller, which may or may not be the world's.

2.6 SimulationLoop: the fixed timestep

The library's position is that a world never runs itself, and `SimulationLoop` does not change that — it is a convenience that calls `World.tick()` for you, and nothing in the engine depends on it. Constructing one starts nothing; `start()` spawns a single daemon thread named `gametools-sim-loop`, which measures elapsed real time, converts it into whole ticks, runs them, and parks for half a tick before measuring again.

The conversion is the interesting part and lives in `advance(realElapsedNanos)`. Elapsed nanoseconds are accumulated; while the accumulator holds at least one tick's worth of time, a tick is run and that much time is subtracted. This is the standard fixed-timestep accumulator, and it gives you a simulation whose step size never varies even though real frames do.

Left unguarded, an accumulator like that has a well-known failure mode. Suppose the process is paused for thirty seconds — a debugger breakpoint, a long garbage-collection pause, a suspended laptop. On resume the accumulator holds six hundred ticks' worth of owed time, the loop tries to run six hundred ticks in one go, that takes longer than the frame it was given, and the debt grows: the spiral of death. `maxCatchUpTicks` caps how many ticks a single `advance` may run — five by default — and any owed time still on the clock after the cap is *discarded*. The simulation therefore loses time rather than falling further behind, which is the correct choice for an authoritative server.

`advance` is public for the caller who already has a loop — a fixed-step server frame, a test harness, an engine tick they do not control — and wants the timestep arithmetic without the thread. It is not safe to call it while the loop

thread is running, or concurrently with itself.

`LoopSettings` is designed to be reconfigured live. Both fields are `@Volatile` and may be written from any thread; because the loop re-reads them every frame and parks in half-tick slices, a change takes effect within one frame. A non-positive or non-finite tick rate is rejected with an exception at the point of assignment, and a catch-up cap below one is silently raised to one.

EVERYTHING RUNS ON THE LOOP THREAD

Both `World.tick()` and your `onTick` callback execute on the loop's thread. The moment the loop is running, the world belongs to it — reading the world from your main thread to build a broadcast is a data race. Do that work inside `onTick`.

2.7 Determinism, and what breaks it

Determinism here means something specific and modest: a world constructed from a given seed, given the same objects in the same order and the same external calls on the same tick numbers, computes the same result every run, on any machine. It does not mean the library records anything. There is no tick-stamped input log and no replay feature; reproducibility is a property you can exploit, not a tool that is handed to you.

It is worth having because it turns two classes of problem from guesswork into engineering. A desync between two servers, or between a server and a re-simulation, becomes bisectable: pin the seed, replay the inputs, find the first tick that differs. And a flaky combat test becomes a stable one, because the evasion roll that failed once will fail every time.

The mechanism is `RandomSource`, a three-method interface — `nextDouble()`, `nextInt(untilExclusive)`, `nextBoolean()` — implemented by `SeededRandom`, a thin wrapper over `kotlin.random.Random`. Everything random the engine does goes through the world's instance. Today that is exactly one thing: the evasion roll in `Alive`. An `Alive` that belongs to no world falls back to a per-instance `SeededRandom` seeded from its own `entityId`, so even worldless combat in a unit test is reproducible.

What breaks it is almost always one of four things. Calling `Math.random()`, `Random.Default` or `UUID.randomUUID()` inside your own subclass or AI puts a choice outside the seeded stream. Iterating a `HashSet` or `HashMap` of objects and acting on the order introduces ordering that is stable within a JVM run but not across them. Applying a player's input at the instant the datagram arrives, rather than at a tick boundary, makes the result depend on network timing. And driving the world from more than one thread makes it depend on the scheduler. Floating-point arithmetic itself is not a hazard here: the JVM's `Double` operations are IEEE-754 and reproducible across platforms, and the library uses only the ordinary operators and `kotlin.math`.

Game Objects

The four-layer entity chain, the movement strategies that drive it, the two projectiles built on it, and how to add a fifth layer of your own.

- 3.1 GameObject — position, activity, the tick hook
- 3.2 VisibleObject — what a renderer needs
- 3.3 Actor — self-propelled motion
- 3.4 The four movement strategies

- 3.5 Alive — health, combat, death
- 3.6 Player and the ownership invariant
- 3.7 Projectiles
- 3.8 Extending the hierarchy

QUICK REFERENCE

FOUR LAYERS · FOUR STRATEGIES · TWO PROJECTILES

GAMEOBJECT — ABSTRACT, THE ROOT

MEMBER	MEANING
location: Point	Position in world space. A <code>val</code> holding a <i>mutable</i> point — mutate it, do not reassign it.
entityId: EntityId	Set once by the owning world. <code>UNASSIGNED</code> until then.
active: Boolean	<code>true</code> . While false, <code>tick()</code> is a no-op.
tick()	Age buffs → <code>onUpdate()</code> → prune expired buffs. Final; do not override.
onUpdate()	OVERRIDE THIS <code>protected open</code> . Call <code>super</code> first.
distanceFrom(other)	<code>infix</code> . <code>Result<Double></code> — fails on a NaN coordinate.
capabilities: Set	What this <i>type</i> can do. Empty here.
can(capability)	Whether it can do it <i>right now</i> , buffs considered.
stats: Map<String, Moddable>	Named stats a buff can reach. Empty here.
buffs: List<Buff>	Read-only view, in application order.
applyBuff / removeBuff	Attach and detach; both fire the buff's hooks.
dispel(name): Int	Remove every buff of that name; returns how many.

VISIBLEOBJECT — OPEN, ADDS DRAWING

MEMBER	MEANING
area: Square	Position and size in one value. Its location is the object's <code>location</code> .
dimensions	Shortcut through to <code>area.dimensions</code> . Default 25×25.
color: Color	Opaque white by default.
texture: String	<code>"default.png"</code> .
angle: Int	Whole degrees CCW from +x. Out-of-range values are normalised and logged.
turns: Boolean	Whether <code>angle</code> means anything to a renderer. <code>false</code> .
visible: Boolean	Whether it is sent to clients. Follows <code>active</code> , but can be set alone.
subObjects: MutableList	Children drawn relative to this one, in insertion order.
collidesWith(other)	<code>open</code> . Centred AABB overlap; touching counts.

Four secondary constructors accept *Dimensions / Point*, *raw sizes*, or *raw coordinates*.

ACTOR — OPEN, ADDS MOTION

MEMBER	MEANING
speed: ModularStat	Units per tick. Base <code>10.0</code> . Exposed to buffs as <code>"speed"</code> .
movement: Movement	<code>Targeting</code> . Assigning it clears the settled flag.
destination: Point	Assigning it also re-aims <code>angle</code> and clears the settled flag.
isAtDestination	Exact equality of the two points.
isOneStepAway	<code>Result<Boolean></code> — within one <code>speed</code> of the destination.
locmod: Point	This tick's displacement vector, length <code>speed</code> .
nearby(quadtree, range)	Broad-phase square window around <code>location</code> . Includes this actor.
capabilities	adds <code>CoreCapability.MOVE</code>

ALIVE — OPEN, ADDS COMBAT

MEMBER	MEANING
health: CombinedStat	Current against a modifiable maximum.
damage / attackTime, attackSpeed / attackRange, evasion	Five <code>ModularStats</code> : <code>10.0</code> , <code>1.7</code> , <code>100.0</code> , <code>750.0</code> , <code>0.0</code> .
world: World?	Set by <code>World.add</code> . Needed for removal on death and for events.
faction: String	<code>"neutral"</code> . The library never compares factions for you.
owner: Player?	Two-way with <code>Player.ownedAlives</code> .
issueAttack(target)	Start the chase-and-swing loop.
cancelAttack()	Stop it from outside. Keeps the current destination.
isAlive	<code>health.current > 0.0</code> .
respawn / deathResponse	Where to return to, and whether to <code>REMOVAL</code> by default.
healthBar	A red sub-object, auto-added, auto-sized, auto-followed.
capabilities	adds <code>CoreCapability.ATTACK</code>

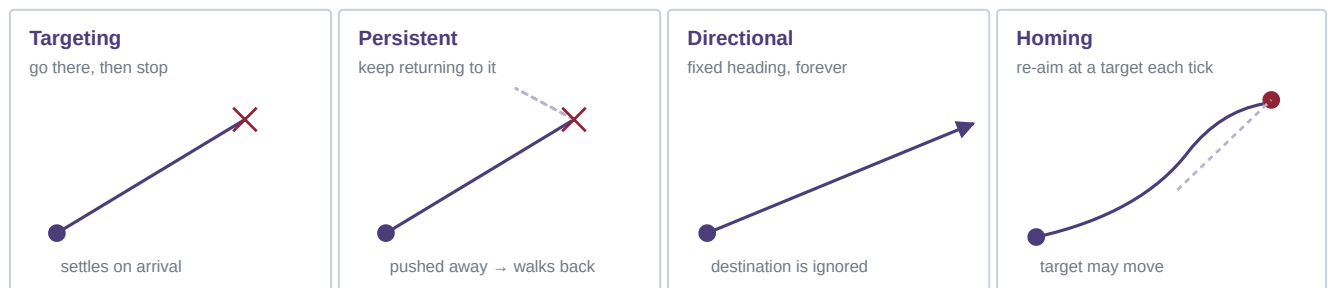


FIGURE 3.1 Every strategy advances the actor at most `speed` units per tick and reports a `Result`.

MOVEMENT — THE SEALED SET

STRATEGY	KIND	PER TICK	USE IT FOR
Targeting	data object	Step towards <code>destination</code> ; snap on arrival; then hold until a new destination is assigned.	Ordinary move orders. The default.
Persistent	data object	Step towards <code>destination</code> unconditionally, every tick.	Anything that should recover from being displaced — a follower, a formation slot.
Directional	data object	Step <code>speed</code> units along <code>angle</code> . <code>destination</code> is not read.	Beams, bullets, drifting scenery.
Homing(target)	data class	Re-point <code>destination</code> at the target's current location, then step towards it.	Guided projectiles, pets, chase AI.

PROJECTILES

TYPE	BEHAVIOUR
Projectile	Abstract. Holds a <code>damage: Double</code> and a <code>protected dealDamageTo(target)</code> . Decides nothing itself.
HomingProjectile	Chases one <code>Alive</code> . On the tick it overlaps that target: deals damage once, sets <code>active = false</code> .
DirectionalProjectile	Flies a fixed heading for <code>maxDuration</code> ticks, damaging every <code>Alive</code> it touches — each at most once. Deactivates when its lifetime runs out.

PROJECTILE DAMAGE IS NOT COMBAT DAMAGE

`dealDamageTo` subtracts from `target.health.current` directly. It does not roll evasion, does not fire the `onDamaged` hooks, does not publish `DamageDealt`, and does not record the attacker — so a kill by projectile produces an `EntityDied` whose `killer` is `null`. If you need attribution or a kill feed, see §3.7 and §6.3.

WHICH SNAPSHOT A BROADCAST OBJECT BECOMES

RUNTIME TYPE	SNAPSHOT	WIRE TYPE TAG
Alive (or a subclass)	<code>AliveSnapshot</code>	"alive"
Actor / Projectile	<code>ActorSnapshot</code>	"actor"
any other <code>VisibleObject</code>	<code>VisibleObjectSnapshot</code>	"visibleObject"
a bare <code>GameObject</code>	Not broadcast at all — only <code>VisibleObject</code> s are.	

IN DETAIL

§3.1 – §3.8

3.1 GameObject — position, activity, the tick hook

Everything the simulation owns is a `GameObject`. The class is deliberately small: a position, a flag, an identity, a per-frame hook, and the two mechanisms — capabilities and buffs — that every layer above builds on.

The position deserves a moment. `location` is declared `val`, but the `Point` it holds is mutable. That is not an accident: the object's position is a stable cell that other things may hold a reference to, and moving the object means writing new coordinates into that cell rather than swapping the cell out. A `VisibleObject` relies on this — its `area.location` and its `location` are the same object, so moving one moves the other. The practical rule is `location.setTo(x, y)` or `location += delta`, never an attempt to reassign.

`active` is the participation switch. While it is `false`, `tick()` returns immediately: no buffs age, no update runs, nothing moves. The property is `open` and `VisibleObject` overrides it to keep `visible` in step, so deactivating a drawable object also stops it being sent to clients. Transitions are logged at `debug`.

The lifecycle split between `tick()` and `onUpdate()` is the library's main extension point and the rule is worth stating twice. `tick()` is not open: it runs the activity check, ages the buffs, calls `onUpdate()`, then prunes expired buffs. `onUpdate()` is `protected open` and empty at the root; every subclass overrides it and calls `super.onUpdate()` as its first statement. Omit that call in a subclass of `Alive` and you have silently disabled movement, death handling and the attack cycle at once.

`distanceFrom` is `infix` and returns `Result<Double>` rather than a bare number, because the underlying geometry reports failure when a coordinate is NaN. That is the house idiom in miniature: an operational failure travels as a value, so the caller decides whether a NaN position is fatal or merely a frame to skip.

3.2 VisibleObject — what a renderer needs

`VisibleObject` adds everything a client needs in order to draw the thing, and nothing else — it does not move. Its state is an `area` (a `Square` pairing position and size), a `color`, a `texture` name, a facing `angle`, and two booleans that a renderer consults: `turns`, meaning “this object's angle is meaningful, draw it rotated”, and `visible`, meaning “send this to clients at all”.

Four secondary constructors exist purely for convenience, accepting a `Dimensions/Point` pair, raw width and height with a point, raw dimensions with coordinates, or four bare numbers, so that the common “just give me a box at a position” case does not require assembling a `Square` by hand.

The angle is stored in whole degrees, counter-clockwise from the positive x-axis, and is normalised on assignment: writing `-90` stores `270`, and writing `450` stores `90`. Out-of-range assignments are normalised and logged at `warn` rather than rejected, on the theory that a caller doing raw angle arithmetic should not have to wrap it themselves — but the warning exists because a stream of them usually means someone is accumulating a rotation without ever wrapping it.

`subObjects` is a list of visible objects drawn relative to this one. The library uses it for exactly one thing — `Alive`'s health bar — but it is a general facility: nameplates, selection rings, status icons and equipment overlays all belong there. Sub-objects are numbered by the world along with their parent, so they are addressable by `EntityId`; they are snapshotted recursively, filtered by their own `visible` flag; and they are *not* ticked independently and *not* inserted into the quadtree. A sub-object that needs to move must be moved by its parent, which is exactly what `Alive.updateHealthBar()` does each frame.

Collision is a centred axis-aligned box test. Each object is treated as a box centred on its `location` with its own `dimensions`, and the two overlap when the gap between their centres is no more than the summed half-extents on both axes. The comparison is inclusive, so boxes that exactly touch are counted as colliding, and a zero-size box can still collide at an exact position match. The method is `open`: override it for circles, capsules or anything else, and every caller in the library — both projectiles included — picks up the new rule, because they all narrow-phase through `collidesWith`.

3.3 Actor — self-propelled motion

An `Actor` is a visible object that moves itself. It contributes three pieces of state — a `speed`, a `destination` and a `movement` strategy — declares the `MOVE` capability, and exposes `speed` to the buff system under the key `"speed"`.

`speed` is a `ModularStat` rather than a `Double` specifically so hastes and slows can coexist without any of them having to know about the others. Change `speed.base` for a permanent change; apply a `StatMod` for a temporary one. The effective value is what every strategy advances by and what `ActorSnapshot` captures. A negative speed is not rejected — it simply runs the actor backwards along its heading, which is occasionally useful and more often a symptom of a slow that stacked further than intended.

Assigning `destination` does three things: it stores the point, clears the internal `settled` flag so a `Targeting` actor resumes moving, and re-aims `angle` at the new point using `atan2`, rounded to whole degrees. Two cases leave the angle alone and say so in the log: a destination containing a NaN coordinate (logged at `warn`) and a destination equal to the actor's current position (logged at `debug`), since neither defines a direction. The angle is computed once, at assignment, and is not re-derived as the actor travels — which matters for `Homing`, where the destination is re-assigned every tick and the angle therefore does track the target.

Three read-only helpers round out the surface. `isAtDestination` is exact point equality — true only after a step has snapped the actor onto its target. `isOneStepAway` reports, as a `Result`, whether the remaining distance is less than `speed`. `locmod` is this tick's displacement: a vector of length `speed` pointing at the destination. Reading `locmod` while the actor is exactly at its destination divides by zero and yields NaN components, so it is only meaningful when there is somewhere to go — the movement code checks first.

Finally, `nearby(quadtrees, range)` is the actor's window onto the spatial index: a square query of half-extent `range` centred on the actor's position. It is a broad phase, so it returns candidates and not answers, and the actor itself is

among them when it is in the tree. The caller supplies the tree and is responsible for its freshness (§2.5).

3.4 The four movement strategies

`Movement` is a sealed class with three `data objects` and one `data class`. Because the stateless three are singletons, assigning a strategy costs nothing and switching one mid-flight is a normal thing to do.

Targeting, the default, is “walk there and stop”. It steps towards the destination, and once the actor arrives it sets an internal settled flag and does nothing on subsequent ticks until a new destination — or a new strategy — clears the flag. This is what you want for a click-to-move order.

Persistent is “walk there, always”. It steps towards the destination every tick with no settled flag, so an actor pushed off its mark walks back. Use it for followers, formation slots and anything that should treat its destination as a position to hold rather than a place to arrive at.

Directional ignores the destination entirely and steps `speed` units along the actor's current `angle`, forever. It is the strategy `DirectionalProjectile` installs on itself.

Homing(target) re-points the destination at the target's current location at the top of every step and then pursues it as `Persistent` would. It holds a hard reference to the target, which has two consequences: the actor keeps chasing a target that has died or left the world (nothing in the strategy checks), and the reference keeps the target object alive for the garbage collector. Replace the strategy when the chase should end.

Every strategy's step returns `Result<Unit>`. `Actor.onUpdate` logs a failure at `error` and continues, so a NaN coordinate costs one frame of movement rather than crashing the tick.

3.5 Alive — health, combat, death

`Alive` is the largest class in the library and the one most worth reading in full. It adds six stats — `health` as a `CombinedStat`, and `damage`, `attackTime`, `attackSpeed`, `attackRange` and `evasion` as `ModularStats` — the `ATTACK` capability, a faction, a world back-reference, an owner, a death response, and an automatically maintained health bar. Its combat behaviour is the subject of chapter 4; what follows is the structure.

The constructor takes a location, dimensions and a maximum health, and requires the last to be positive — `CombinedStat` throws otherwise. All six stats are exposed through the `stats` map under their own names, so a buff

can address any of them by string.

`world` is a nullable back-reference set by `World.add`. It is not decoration: it is how a dying actor reaches `removeList`, how combat events reach the bus, and how the evasion roll reaches the seeded RNG. An `Alive` with a null world still fights, but silently — no events are published — and cannot remove itself on death.

`faction` is a plain string, defaulting to `"neutral"`. The library never reads it: nothing checks factions before allowing an attack, and friendly fire is entirely possible. It exists so your targeting code has somewhere canonical to put the answer.

The health bar is the one piece of presentation the class builds for you. It is a red `VisibleObject`, twenty per cent of the actor's height, added to `subObjects` in the constructor, and refreshed every tick: its position is snapped back onto the actor and its width is scaled to the current health fraction, clamped to `0..1`. Two details are deliberate. Its geometry is built from fresh `Dimensions` and `Point` instances rather than the constructor's arguments, so resizing the bar can never mutate the actor's own geometry — those objects are shared. And its full width and vertical offset are captured once, from the actor's size at construction, so an actor that is later resized keeps a bar of its original width.

DON'T WANT A HEALTH BAR?

Remove it — `subObjects.remove(healthBar)` — or set `healthBar.visible = false`. The per-tick refresh still runs either way; it is cheap, but it is not skippable without overriding `onUpdate`.

3.6 Player and the ownership invariant

`Player` is a game-domain owner: a name and a roster of `Alive` actors. It is emphatically *not* the networking notion of a player — `GameServer` tracks connections by name and knows nothing about rosters. Keeping the two separate is intentional; a match may have AI players with no connection, and a connection may control no units.

The interesting part is the invariant. `Alive.owner` and `Player.ownedAlives` are kept in step in both directions, and the setter on `owner` is the single place that enforces it: assigning a new owner removes the actor from the previous owner's roster and adds it to the new one, and assigning `null` simply removes it. `Player.own` and `Player.disown` are conveniences that drive that same setter and report whether anything changed. The two roster mutators on `Player` are `internal` precisely so no other code can break the invariant. The upshot is that an `Alive` is on at most one roster at any moment, and transferring a unit between players is a single assignment.

`ownedAlives` returns the roster in acquisition order; `livingAlives` filters it by `isActive` on every read, which is convenient for a defeat condition and worth caching if you call it per tick over a large roster.

3.7 Projectiles

`Projectile` is an abstract `Actor` carrying a `damage` value and one `protected` method, `dealDamageTo(target)`, which subtracts that damage from the target's current health and logs it. It decides nothing about what to chase or how to recognise a hit; a subclass supplies both.

`HomingProjectile` is constructed with a target and a quadtree. It installs `Movement.Homing` on itself, and on every tick after moving it asks whether it has run into its target. The test is two-phase: a broad-phase `nearby` query over the quadtree, with a radius derived from the two objects' summed half-extents, then a narrow-phase `collidesWith`. On a hit it deals its damage once, latches an internal flag so it cannot hit twice, and sets `active = false` — which, through the `VisibleObject` override, also clears `visible`, so a spent projectile stops both simulating and appearing on clients.

`DirectionalProjectile` is constructed with a heading, a lifetime in ticks, a quadtree and an optional search radius that defaults to three times its own width. It sets its angle, installs `Movement.Directional`, and on every tick gathers the `Alives` within the search window that it currently overlaps, damaging each one that it has not damaged before — a set of already-hit actors is what allows it to pierce a line of targets while hitting each once. When its tick count reaches the lifetime it deactivates. The constructor rejects a non-positive lifetime by throwing.

FAST PROJECTILES TUNNEL

Both projectiles move first and test for overlap afterwards, and the test is a point-in-time box overlap with no swept volume. A projectile whose `speed` exceeds the combined size of itself and its target can step clean over it between ticks and never register a hit. Keep `speed` below roughly the summed half-extents, enlarge the projectile's `dimensions`, or raise the tick rate.

The second thing to know about both is that their damage path bypasses combat entirely. `dealDamageTo` writes to `health.current` directly: no evasion roll, no `onDamaged/onHitBy` hooks, no `DamageDealt` event, and no record of who fired. A target killed by a projectile still dies correctly — the death check runs on the target's own next tick and publishes `EntityDied` — but the `killer` field is `null`. If your game needs projectile kills attributed, subclass the projectile, carry the firer, and publish your own event or route the damage through the owner instead.

Neither projectile removes itself from the world. A spent projectile is inactive and invisible but still in `gameObjects`, still holding an `EntityId`, and still being inserted into the quadtree every tick. Over a long match that accumulates. Sweep them — a periodic pass that queues inactive projectiles into `removeList`, or a subclass that queues itself the tick it spends.

3.8 Extending the hierarchy

Adding your own entity type is the normal way to use this library. The checklist is short.

Subclass at the shallowest layer that fits. A trigger volume with no appearance is a `GameObject`. Scenery is a `VisibleObject`. Something that moves but cannot be killed is an `Actor`. Only reach for `Alive` when the thing genuinely has health and fights, because every `Alive` pays for a death check, a health-bar refresh and an attack-state evaluation on every tick.

Override `onUpdate`, call `super` first. This is the one rule that silently breaks everything when forgotten.

Declare capabilities as an extension of the supertype's. Write `override val capabilities = super.capabilities + MyCapability.CAST`, and gate your per-tick work on `can(MyCapability.CAST)` so that a buff can suppress it the same way the built-in layers allow. Your capability needs a stable, unique `id` string, because that `id` is what crosses the wire in a `BuffSnapshot`.

Expose new stats through the `stats` map. Write `override val stats get() = super.stats + ("mana" to mana)` — as a getter, not a stored property, so it keeps tracking a stat you later reassign wholesale. Anything not in that map is invisible to buffs, and a buff naming a missing key is logged and skipped rather than failing loudly.

Publish events for things other systems care about. Reach the bus through `world?.events` if your type extends `Alive`, or hold your own reference otherwise. Today `GameEvent` is a sealed interface in the library's own module, so your own event types cannot join it; publish through your own bus, or use one of the existing events where the fit is honest.

A WORKED EXAMPLE

§10.8 builds a `Caster` subclass end to end: a new capability, a new stat, a gated per-tick behaviour, and a buff that silences it.

Stats, Buffs & Combat

Numbers that other numbers can bend, effects that bend them for a while, and the swing-by-swing machinery that turns them into a fight.

4.1 ModularStat and the value formula

4.2 StatMod: type × stacking

4.3 CombinedStat

4.4 Moddable and the stats map

4.5 Capabilities

4.6 Buffs

4.7 The attack cycle

4.8 Evasion, damage and attribution

4.9 Death, respawn and removal

4.10 The hook catalogue

QUICK REFERENCE

THE FORMULA · THE MATRIX · THE STATE MACHINE

value = base × (1 + Σ additively-stacking factors) × Π multiplicatively-stacking factors + Σ flat amounts



Recomputed eagerly on every applyMod / removeMod that changes the mod set — reading .value is free.

FIGURE 4.1 The whole of `ModularStat`’s arithmetic. With no mods applied the two multiplicative terms are `1.0` and the additive term is `0.0`, so `value == base`.

STATMOD — TYPE × STACKING, AND WHAT EACH COMBINATION DOES

STACKINGTYPE	WITH TYPE.ADDITIVE	WITH TYPE.MULTIPLICATIVE	DEFAULT TYPE
NONE	Inert. <code>applyMod</code> does not even store it.		
SETTING	Replaces the value of an applied mod of the same name; otherwise added. Counts toward the flat total.	Stored, but contributes nothing to <code>value</code> .	
INCREMENTING	Adds its value onto an applied mod of the same name; otherwise added. Counts toward the flat total.	Stored, but contributes nothing to <code>value</code> .	
ADDITIVE DEFAULT	Always added as its own entry. Counts toward the flat total.	Summed with its peers into one bonus on top of <code>1.0</code> . <code>0.25</code> means +25 %.	
MULTIPLICATIVE	Always added as its own entry. Counts toward the flat total.	Multiplied with its peers as a factor. <code>1.25</code> means ×1.25.	

Note the two red cells: `SETTING` and `INCREMENTING` are meaningful only on an `ADDITIVE` mod. See §4.2.

READING A MOD'S VALUE

- `Type.ADDITIVE` — a flat amount, in the stat's own units. `+15` damage.
 - `Type.MULTIPLICATIVE + ADDITIVE` stacking — a *bonus fraction*. `0.25` = +25 %.
 - `Type.MULTIPLICATIVE + MULTIPLICATIVE` stacking — a *factor*. `1.25` = $\times 1.25$.
- `StatMod("x", 0.25)` with no further arguments is the middle case: +25 %.

REMOVAL IS BY NAME, NOT BY INSTANCE

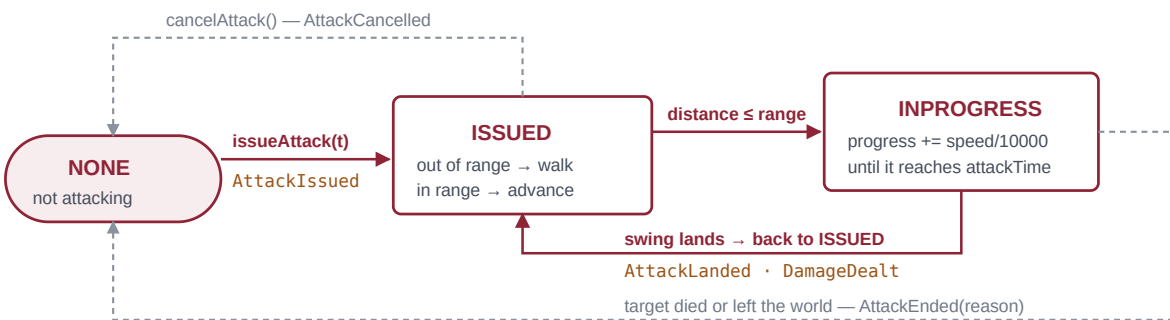
`removeMod(mod)` drops every applied mod whose `name` matches — including ones applied by somebody else. Two concurrent buffs that both write a mod called `"haste"` onto `speed` share fate: whichever expires first removes both. Give concurrent effects distinct mod names.

COMBINEDSTAT

MEMBER	MEANING
<code>CombinedStat(start, max)</code>	Starts at <code>start</code> against a ceiling of <code>max</code> . Throws if the ceiling is not positive.
<code>CombinedStat(start)</code>	Starts full: both current and ceiling at <code>start</code> .
<code>max: ModularStat</code>	The ceiling, itself modifiable.
<code>current: Double</code>	Free to sit anywhere, including at or below zero.
<code>fractionOfMax</code>	<code>current / max</code> . <code>1.0</code> when full; free to leave <code>0..1</code> .
<code>applyMod / removeMod</code>	Adjust <code>max</code> and rescale <code>current</code> so the fraction is preserved.

WHY THE RESCALE MATTERS

A unit at 50/100 health gains a +100 % max-health buff. Without the rescale it would sit at 50/200 — the buff would have *halved* its effective condition. `CombinedStat` captures the fraction first, moves the ceiling, then restores the fraction: 100/200. When the buff expires the same arithmetic runs in reverse and it is back at 50/100.



The target check runs at the top of every tick, from either active state. A swing already in progress is not re-range-checked until it lands.

FIGURE 4.2 `Alive`'s attack cycle. Three states, two exits. `cancelAttack()` is the caller's exit; `AttackEnded` is the simulation's.

TIME TO A SWING

170 ticks

AT STOCK VALUES

`attackTime / (attackSpeed / 10000)`
— that is 8.5 s at 20 Hz. Raise `attackSpeed` to speed it up.

BUFF DURATION

exactly n

UPDATES IN FORCE

A buff with `durationTicks = n` has its `onTick` fired n times and is removed at the end of the nth. A negative duration never expires.

DEATH LATENCY

1 tick

WORST CASE

Death is detected on the victim's own next update, not when the damage lands — so a corpse can still be targeted for the rest of the frame.

CAPABILITIES

CAPABILITY	ID	SUPPRESSING IT MEANS
CoreCapability.MOVE	"move"	The actor holds position — its <code>Movement</code> is not stepped this tick. A <i>root</i> .
CoreCapability.ATTACK	"attack"	The attack order is kept but swing progress freezes. A <i>disarm</i> . Death handling and the health bar are unaffected.

Suppress both at once for a *stun*. A capability stays suppressed while *any* active buff lists it, so overlapping suppressors resolve themselves.

IN DETAIL

§4.1 – §4.10

4.1 ModularStat and the value formula

A game is mostly numbers that other things want to change, and the naive representation — a `Double` that a buff multiplies on the way in and divides on the way out — falls apart the moment two effects overlap, because floating-point division does not reliably undo multiplication and because removal order starts to matter. `ModularStat` solves this by never mutating the underlying number at all. It keeps a `base` and a list of named modifiers, and derives the effective `value` from both.

The derivation is the formula in Figure 4.1: the base is scaled by the combined multiplicative factor and then the flat additive total is added on top. Order matters and is fixed — percentage changes apply to the base, flat changes apply afterwards, so a `+50` flat bonus is not itself amplified by a `+20 %` buff. Recomputation is eager: every `applyMod` and `removeMod` that actually changes the mod set recalculates `value` immediately, so reading it is a field access and can be done freely inside a tick.

The second half of the type's job is ergonomic. `ModularStat` defines `plus`, `minus`, `times`, `div`, `compareTo` and the unary operators against both `Double` and another `ModularStat`, and top-level extension operators supply the mirror image so that `3.0 * stat` resolves as readily as `stat * 3.0`. Its `toString` prints the effective value. The result is that a stat can stand in for a

number nearly everywhere — `if (distance > attackRange)` and `location += Point(speed * cos(r), speed * sin(r))` are both real lines from the library, with no `.value` in sight.

THE ONE PLACE THE OPERATORS DO NOT REACH

Kotlin resolves `compareTo` for `<` and friends, but equality is not an operator you can supply this way: `stat == 10.0` does not compile, and `stat == otherStat` is reference equality. Compare `.value` explicitly when you need equality, and remember that comparing floating-point numbers for exact equality is usually the wrong test anyway.

4.2 StatMod: type × stacking

A `StatMod` carries four things: a `name`, a `value`, a `type` and a `stackingType`. The name is the identity — it is what `SETTING` and `INCREMENTING` fold against, and it is what `removeMod` matches on. The type says *where in the formula* the value lands: `ADDITIVE` contributes to the flat total, `MULTIPLICATIVE` contributes to the scaling factor. The stacking type says *how repeated mods combine*.

Two of the stacking types govern how a mod is stored. `SETTING` overwrites the value of an already-applied mod of the same name, which is how you express “this effect refreshes rather than stacks”. `INCREMENTING` adds onto it, which is how you express a stacking debuff that keeps one entry with a growing magnitude. `NONE` is inert — applying it stores nothing and changes nothing, which is occasionally useful as a placeholder. The other two always add a fresh entry and instead govern how the entries *combine*: `ADDITIVE` stacking sums the bonuses, so `+20 %` and `+10 %` give `+30 %`; `MULTIPLICATIVE` stacking multiplies the factors, so `×1.2` and `×1.1` give `×1.32`. Additive stacking is the familiar behaviour of most RPG gear; multiplicative stacking is what you want for effects that should compound but never quite reach 100 % of anything.

The interaction between the two axes has one sharp edge, and it is the most likely source of a “my buff does nothing” bug. The multiplicative half of the formula only consults mods whose stacking type is `ADDITIVE` or `MULTIPLICATIVE`. A mod that is `Type.MULTIPLICATIVE` with `StackingType.SETTING` or `INCREMENTING` is stored in the list, is removable by name, and contributes *nothing* to the computed value. In practice: **use `SETTING` and `INCREMENTING` only on `ADDITIVE` mods.**

The defaults reward the common case. `StatMod("sword-of-power", 0.25)` — name and value only — is a multiplicative, additively-stacking mod, which is to say a plain `+25 %`. A flat bonus needs the type spelled out: `StatMod("blessing", 15.0, Type.ADDITIVE)`.

Removal deserves repeating because it is the other common surprise. `removeMod` takes a `StatMod` but uses only its name, and it removes *every* applied mod with that name. Two independent buffs that each apply a mod called "haste" to the same stat are therefore entangled: when the first expires, both disappear, and the second buff's later expiry is a no-op. Namespace your mod names — include the buff's identity, or the caster's — whenever two instances of an effect can coexist.

4.3 CombinedStat: current against a moving ceiling

Health, mana and stamina are not one number but two: an amount and a ceiling, where the ceiling itself can be buffed. `CombinedStat` is that pair. Its `max` is a full `ModularStat`, so everything in §4.1 and §4.2 applies to the ceiling; its `current` is a plain mutable `Double`, deliberately unconstrained so that it can sit at or below zero and let callers test for depletion rather than having to intercept the moment of it.

What the type adds is the rescaling rule. When a mod is applied to or removed from the ceiling, the stat first captures `current / max`, then changes the ceiling, then restores that fraction against the new ceiling. A unit at half health that gains a +100 % maximum-health buff ends up at half health of the new maximum, not at a quarter; when the buff drops, it is back where it started. This is almost always the behaviour a designer means, and getting it wrong is a classic source of buffs that feel like debuffs.

Construction is through one of two public secondary constructors — a starting value with an explicit ceiling, or a single value used for both — because the primary constructor is private. A non-positive ceiling throws at construction. Note that this validates only the initial value: a debuff that drags the modified maximum to zero or below is not caught, and `fractionOfMax` will produce an infinity or a NaN. Guard your ceiling debuffs.

Like `ModularStat`, `CombinedStat` carries the full operator set — this time delegating to `current` — so `if (health < 50.0)` reads naturally. Direct assignment to `current` is the ordinary way to heal or to apply damage from outside combat; be aware that doing so bypasses every hook and every event (§4.8).

4.4 Moddable and the stats map

`Moddable` is a two-method interface — `applyMod` and `removeMod` — implemented by both stat types. Its purpose is to let a `Buff` hold a map of stat keys to modifiers without knowing or caring which concrete kind of stat sits behind each key.

The map itself is `GameObject.stats`, an `open val` that each layer extends: the root is empty, `Actor` adds `"speed"`, and `Alive` adds `"health"`, `"damage"`, `"attackTime"`, `"attackSpeed"`, `"attackRange"` and `"evasion"`. The declaration is a getter rather than a stored property, which matters: it means the map is rebuilt on each read and therefore keeps pointing at the current object if a stat property is reassigned wholesale rather than mutated. Follow the same pattern in your own subclasses — `override val stats get() = super.stats + ("mana" to mana)`.

A buff naming a key the object does not expose is logged at `warn` and that one modifier is skipped; the rest of the buff still applies. This is forgiving by design — a generic “battle blessing” can carry mods for stats that only some entity types have — but it does mean a typo in a key name fails quietly. Watch the log the first time you write a new buff.

4.5 Capabilities

A `Capability` is a named thing an object type is able to do, expressed as an interface with a single stable `id` string. The built-in hierarchy supplies two through the `CoreCapability` enum: `MOVE`, declared by `Actor`, and `ATTACK`, declared by `Alive`.

Two questions are kept separate. `capabilities` answers “what can this *type* do, in principle?” and is a compile-time fact about the class. `can(capability)` answers “can this *object* do it right now?” and is true only when the capability is in the type's set and no currently active buff lists it among its suppressed capabilities. Each layer gates its own per-tick work on the second question: `Actor.onUpdate` only steps its movement while `can(MOVE)`, and `Alive.onUpdate` only advances the attack cycle while `can(ATTACK)`.

That single mechanism covers the whole crowd-control vocabulary without adding a field anywhere. A root is a buff suppressing `MOVE`; a disarm suppresses `ATTACK`; a stun suppresses both. Because `can` asks whether *any* active buff suppresses the capability, overlapping suppressors compose correctly — two stuns with different durations leave the target stunned until the later one expires, with no reference counting on your part. And because suppression freezes rather than cancels, a disarmed unit keeps its attack order and its accumulated swing progress and resumes exactly where it left off.

Your own capabilities are ordinary implementations of the interface — usually an enum, as `CoreCapability` is. The only real constraint is that the `id` must be stable and unique across every capability in your game, because that string is what crosses the wire inside a `BuffSnapshot` so a client can grey out a disabled ability.

4.6 Buffs

A `Buff` bundles three things and a lifetime: a name, a map from stat key to `StatMod`, a set of capabilities to hold suppressed, and a duration in ticks. Attach one with `GameObject.applyBuff(buff)` and the object drives it from there.

Application is immediate and ordered: the buff is appended to the object's list, each of its modifiers is applied to the matching stat (skipping and logging any key the object does not expose), and `onApplied` fires. Apply buffs after construction — the object's `stats` and `capabilities` must already be in place.

The countdown happens inside `GameObject.tick()`, before `onUpdate()`: every active buff has `onTick` fired and, if its duration is positive, decremented. After `onUpdate()`, any buff that has reached zero is removed — reverting its modifiers, releasing its suppressions and firing `onExpired`. The arithmetic works out to the clean rule that a buff created with `durationTicks = n` is in force for exactly n updates and fires `onTick` exactly n times. A *negative* duration never counts down, which is how you express an aura or a permanent effect that is removed explicitly.

There are three ways off: expiry, `removeBuff(buff)` for a specific instance, and `dispel(name)`, which removes every attached buff with that name and returns how many it removed. All three take the same path, so the hooks and the reversion always run.

Subclassing is how a buff gets behaviour beyond arithmetic. Override `onApplied` to spawn a visual sub-object, `onTick` for damage or healing over time, and `onExpired` to clean up. A poison is a buff whose `onTick` subtracts from the target's health; a shield is a buff whose `onApplied` raises a maximum and whose `onExpired` lowers it again — although for the latter a `statMod` on `"health"` does the job with no code at all.

STACKING IS THE CALLER'S PROBLEM

The library does not deduplicate buffs. Applying the same buff object twice attaches it twice. Applying two buffs whose modifiers share a `StatMod` name on the same stat entangles their removal (§4.2). If your game has a notion of “refresh instead of stack”, implement it at the point of application: `dispel(name)` first, then `applyBuff`.

On the wire, a buff appears as a `BuffSnapshot` carrying its name, its remaining duration and the ids of the capabilities it is suppressing — enough for a client to draw a timer and grey out a disabled button, and deliberately not enough to expose the modifier values.

4.7 The attack cycle

`Alive`'s combat is a three-state machine advanced once per tick, and everything it does is driven from `onUpdate`. The states are `NONE`, `ISSUED` and `INPROGRESS`.

`issueAttack(target)` moves the machine to `ISSUED`, records the target, fires `onAttackIssued` on the attacker and `onTargetedByAttack` on the target, and publishes `AttackIssued`. Nothing else happens synchronously — the first actual progress is on the next tick.

From then on, each tick begins with a viability check: if the target has died or has left the world, the attack ends itself (§4.7.1). Otherwise, in `ISSUED`, the attacker measures the distance to its target. If that exceeds `attackRange`, the attacker assigns the target's location as its own destination and walks; if not, it advances to `INPROGRESS`. A distance measurement that fails — a NaN coordinate — is logged and treated as zero, which is to say as in-range, so the actor keeps engaging rather than freezing.

In `INPROGRESS` the attacker accrues swing progress at `attackSpeed / 10000` per tick. When progress reaches `attackTime`, the swing lands: progress resets to zero, the state returns to `ISSUED`, and one swing is resolved. The loop then repeats indefinitely — the attack does not stop after one hit.

Two things about the timing are worth internalising. The first is the magic constant: the divisor of ten thousand means `attackSpeed` is not “swings per second” or anything else intuitive, and the stock numbers produce one swing every 170 ticks. The second is that the range check happens only in `ISSUED`. Once a swing is under way, the target can walk out of range and the swing still lands; the range is re-tested only when the machine returns to `ISSUED` afterwards.

AN ALIASING DETAIL

While closing, the attacker assigns `destination = target.location` — the target's own `Point` instance, not a copy. The attacker's destination therefore tracks the target implicitly for as long as that assignment stands. It is re-assigned every tick anyway, so the behaviour is the same either way, but it means holding a reference to an attacker's `destination` can leave you holding the target's live position.

4.7.1 The two exits

An attack stops in one of two ways, and the library reports them differently on purpose.

`cancelAttack()` is the caller's exit: an order to disengage, retreat or retarget. It clears the state, fires `onAttackCancelled` and publishes `AttackCancelled` with the former target. It deliberately leaves the attacker's current

`destination` alone — cancelling an attack is not a move order, and if the unit should also go somewhere you issue that separately. Calling it when no attack is in progress is a no-op.

`AttackEnded` is the simulation's exit: the target can no longer be fought. The check runs at the top of every tick from either active state and produces one of two reasons — `TARGET_DIED` when the target's health has run out, or `TARGET_REMOVED` when it has left its world while still alive. It fires `onAttackEnded(reason)` and publishes `AttackEnded`. The removal test is careful to fire only for a target that *once* belonged to a world and no longer does, so worldless combat in a unit test is unaffected.

Before 3.1 neither exit existed, and an attacker whose target died would walk to the corpse's last position and swing at nothing forever. If you are porting from an older version, that is the behaviour change to look for.

4.8 Evasion, damage and attribution

A landing swing resolves through a short chain, and each link has a hook on both sides. First `onAttack` fires on the attacker and `onAttacked` on the target. Then evasion is rolled: a value is drawn from the world's seeded RNG and compared with the target's `evasion`, and if the roll does not exceed it the swing is dropped entirely — silently, with no event and no hook. Then `onHitting` and `onHitBy` fire, damage is applied, and `AttackLanded` is published.

Damage application itself runs `onDamaging` on the attacker and `onDamaged` on the target, passes the raw amount through a mitigation step — which in the base library is the identity function, and is the natural place for a subclass to introduce armour or resistances — records the attacker as the target's *last damager*, subtracts from `health.current`, and publishes `DamageDealt`. A computed amount of exactly zero short-circuits before any of that, so a fully mitigated hit records nothing.

The evasion roll uses the world's `rng` when the attacker belongs to a world, and otherwise falls back to a per-instance `SeededRandom` seeded from the attacker's `entityId` — so combat in a worldless unit test is still reproducible rather than merely arbitrary.

THERE IS NO PUBLIC WAY TO DEAL DAMAGE

`takeDamage` is private. The only route into the hook-and-event path is a real swing from another `Alive`. Anything else — a projectile, an environmental hazard, a scripted execution — has to write `health.current -= x` directly, which fires no hooks, publishes no `DamageDealt`, and records no last damager, so the resulting `EntityDied` carries a `null` killer. If your game needs uniform attribution, wrap damage in a function of your own that also publishes what you need.

A DOCUMENTATION DISCREPANCY WORTH KNOWING

The KDoc on `GameEvent.AttackLanded` says the event fires before evasion is rolled, and that “a landed swing is not necessarily a hit”. The implementation publishes it *inside* the hit path, after damage has been applied — so in the shipped code an evaded swing produces no event at all. Trust the behaviour described here; treat the KDoc sentence as stale.

4.9 Death, respawn and removal

An `Alive` is alive while `health.current` is strictly above zero. Death is not detected at the moment damage lands; it is detected at the top of the victim's own next update, when `contemplateLife` notices the depleted health and calls `die()`. That means a freshly killed unit remains in the world, targetable and visible, until its turn comes round — within the same tick if it sits later in the object list than its killer, otherwise on the following one.

`die()` is guarded so that one death produces one response, and then applies the actor's `deathResponse`. `REMOVAL`, the default, queues the actor into its world's `removeList`, so it is dropped at the end of the tick and an `EntityRemoved` follows. An actor with no world logs a warning and stays in play — the single most common symptom of having added it with `gameObjects +=` instead of `add()`. `RESPAWN` moves the actor to its `respawn` point, which defaults to wherever it was created, restores full health, and re-arms the guard so it can die again later.

Either way, `onDeath()` fires and `EntityDied` is published, carrying the actor and its last damager as the killer — `null` when nothing recorded one, which covers projectile kills and direct health edits alike. Once the actor is alive again, the guard and the recorded killer are both cleared on the next update.

`die()` is `protected open`, so a subclass can replace the response entirely — dropping loot, splitting into two smaller units, starting a death animation before queueing removal. Call `super.die()` unless you genuinely intend to suppress the standard behaviour.

4.10 The hook catalogue

Every extension point on `Alive` and `Buff`, in the order a full swing fires them. All are `protected open` on `Alive` and public `open` on `Buff`, and all do nothing by default.

HOOKS, IN FIRING ORDER

HOOK	FIRES ON	WHEN
<code>onAttackIssued()</code>	attacker	The moment <code>issueAttack</code> is called.
<code>onTargetedByAttack()</code>	target	Same moment, on the other side. Useful for aggro.
<code>onAttack(target)</code>	attacker	Start of each swing, before evasion.
<code>onAttacked(attacker)</code>	target	Same moment, on the other side.
<code>onHitting(target)</code>	attacker	The swing was not evaded.
<code>onHitBy(attacker)</code>	target	Same moment, on the other side.
<code>onDamaging(target)</code>	attacker	Immediately before health is subtracted.
<code>onDamaged(attacker)</code>	target	Same moment, on the other side.
<code>onAttackCancelled()</code>	attacker	<code>cancelAttack()</code> was called.
<code>onAttackEnded(reason)</code>	attacker	The target died or left the world.
<code>die()</code>	victim	<code>open</code> — the whole death response. Call <code>super</code> .
<code>onDeath()</code>	victim	Once, after the death response has been applied.
<code>onUpdate()</code>	any object	Every tick while active. Call <code>super</code> first.
<code>Buff.onApplied(target)</code>	buffed object	Right after the buff's mods and suppressions take effect.
<code>Buff.onTick(target)</code>	buffed object	Once per tick, before the duration is decremented.
<code>Buff.onExpired(target)</code>	buffed object	Right after the buff is removed and reverted.
<i>Hooks run synchronously inside the tick. Keep them short, and never block in one.</i>		

Spatial Indexing

One data structure, four methods, and the broad-phase discipline that keeps proximity questions from costing $O(n^2)$.

- 5.1 What a point-region quadtree is
- 5.2 The four operations
- 5.3 Query semantics: the half-open box

- 5.4 Shape, balance and rebuilding
- 5.5 Broad phase and narrow phase
- 5.6 Limits and traps

QUICK REFERENCE

QUADTREE<N, E>

QUADTREE<N : COMPARABLE<N>, E>

OPERATION	BEHAVIOUR
<code>insert(x, y, element)</code>	Place <code>element</code> at the point. An emptied slot on the path is reused before a new node is created.
<code>retrieveBox(minX, minY, maxX, maxY): List<E></code>	Every element whose point lies in the <i>half-open box</i> $(\text{minX}, \text{maxX}] \times (\text{minY}, \text{maxY}]$, in unspecified order.
<code>remove(x, y, element)</code>	Empties the first node on the path holding that exact instance. The node stays as a reusable slot.
<code>clear()</code>	Drops the whole tree.

In the engine, N is always Double and E is always VisibleObject — but the type is fully generic.

ORIENTATION

North is `+y`, matching the rest of the engine. A node splits the plane at its own point, not at a cell midpoint.

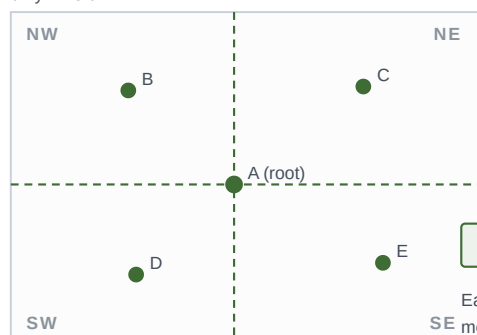
COST

Insert and query are $O(\log n)$ on well-spread data and $O(n)$ in the worst case. The tree is unbalanced — insertion order alone decides its shape.

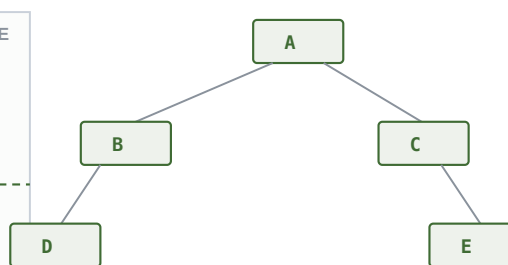
NOT THREAD-SAFE

No synchronisation of any kind. One thread, like everything else in the core.

THE PLANE

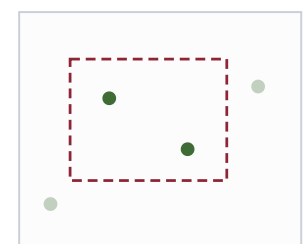


THE TREE THAT MAKES



Each node owns one point and four quadrants measured from that point. Shape follows insertion order.

A QUERY



`retrieveBox` returns the two points inside the box. It never sorts them, and it tests points, never extents.

FIGURE 5.1 The tree indexes *points*. An object's size plays no part in it — that is what the narrow phase is for.

THE IDIOM

Rebuild once per frame, then query freely: `clear()`, insert everything at its current position, and let every object's tick read a consistent index. `World.tick()` does exactly this in phase 2.

THREE SHARP EDGES

- `remove` leaves a tombstone that the next insert along that path takes over *at the old node's coordinates* · the query box is half-open, so a point exactly on the low edge is excluded
- a query returns candidates whose *centre* is inside the box, not objects that overlap it.

5.1 What a point-region quadtree is

The question a game asks constantly is “what is near this?” Answering it by scanning every object is $O(n)$ per question and therefore $O(n^2)$ per frame, which is fine for twenty objects and ruinous for two thousand. A spatial index trades a little memory and a rebuild cost for the ability to answer the same question by visiting only the part of the plane that can contain an answer.

GameTools uses a *point-region* quadtree, sometimes called a point quadtree. Each node stores one point and its associated element, and divides the rest of the plane into four quadrants measured *from that point* — not from the midpoint of a fixed cell, which is what a region quadtree would do. Insertion walks down from the root, choosing a quadrant at each node by comparing coordinates, until it reaches an empty spot. Querying walks down the same way, pruning any subtree whose quadrant cannot intersect the query box.

North is the `+y` direction, consistently with the rest of the engine. The quadrant chosen for a point `(x, y)` at a node `(nx, ny)` is north-west when `x < nx` and `y ≥ ny`, north-east when both are at least the node's, south-east when `x ≥ nx` and `y < ny`, and south-west when both are below. Ties therefore lean north and east.

The implementation is generic in both the coordinate type — anything `Comparable` — and the element type. The engine only ever instantiates it as `Quadtree<Double, VisibleObject>`, but nothing stops you using it for an integer tile grid or for your own non-entity data.

5.2 The four operations

`insert(x, y, element)` walks the quadrant path from the root and places the element where the path runs out, creating a node. One special case matters: if it encounters a node whose element has been cleared by `remove`, it reuses that empty slot instead of continuing. This keeps the tree from filling up with dead nodes, at a cost described in §5.6.

`retrieveBox(minX, minY, maxX, maxY)` collects every element whose indexed point lies in the box and returns them as a list in unspecified order. Note the argument order — both minimums, then both maximums — which is easy to transpose. The traversal prunes: at each node it works out which of the four quadrants can possibly intersect the box and descends only into those.

`remove(x, y, element)` walks the same path looking for a node whose element is that exact instance, compared by reference identity rather than `equals`. When it finds one it sets the element to `null`, leaving the node in place so its children remain reachable. If the element is not on that path — because the coordinates given do not match the ones it was inserted at, for instance — nothing happens and nothing is reported.

`clear()` drops the root and therefore the whole tree, in constant time. This is the operation the per-frame rebuild uses.

5.3 Query semantics: the half-open box

The inclusion test is worth stating precisely, because it is not symmetric. A point is returned when

$$\text{minX} < x \quad \text{and} \quad \text{minY} < y \quad \text{and} \quad \text{maxX} \geq x \quad \text{and} \quad \text{maxY} \geq y$$

— that is, the box is half-open, excluding its low edges and including its high ones: `(minX, maxX] × (minY, maxY]`. An object sitting exactly on the left or bottom edge of your query window will not be returned; one sitting exactly on the right or top edge will.

In practice this bites in exactly one situation: a query centred on an object, with a radius chosen so that a second object sits precisely on the boundary. Because `Actor.nearby(quadtrees, range)` builds its window as `location ± range` on both axes, an object at exactly `location.x - range` is missed. The fix is trivial — widen the range slightly — and the situation is rare with floating-point positions, but it is real and it is the sort of thing that produces an intermittently missed collision.

The other thing to hold onto is that the query tests *points*. An object whose centre is outside the box is not returned even if its bounding box overlaps the box substantially. When you size a broad-phase window, add the largest half-extent you expect to meet — which is precisely what `HomingProjectile` does, deriving its search radius from the summed half-extents of itself and its target.

5.4 Shape, balance and rebuilding

This quadtree does not balance itself. The first point inserted becomes the root and divides the plane; every subsequent point is placed relative to the points already there. Insert points in a random order and the tree is roughly logarithmic in depth. Insert them in sorted order — all along a line, left to right — and every point lands in the same quadrant of the previous one, producing a degenerate chain with $O(n)$ lookups. Insert several objects at the identical coordinate and they chain north-east from one another for the same reason.

The library's answer to all of this is not to rebalance but to rebuild. `World.tick()` clears the tree and re-inserts every visible object at the top of every frame, in `gameObjects` order. Rebuilding is $O(n \log n)$ on well-spread data and, more importantly, is trivially correct: there is no incremental-update code to get wrong when an object moves, and no possibility of the index disagreeing with reality. For the scale GameTools targets — up to a few thousand entities at 10–20 Hz — that is the right trade.

If you find yourself with a pathological distribution, the lever available to you is the order in which objects sit in `gameObjects`, since that is the insertion order. A world populated by streaming in a sorted map file will build a worse tree than one populated in spawn order.

5.5 Broad phase and narrow phase

Every proximity question in the library is answered in two stages, and adopting the same discipline in your own code is the single most useful thing this chapter has to offer.

The **broad phase** is cheap, approximate and inclusive: ask the quadtree for everything whose centre lies within a generous window. It may return objects that are not really in contact; it must not miss any that are. The **narrow phase** is exact and expensive: for each candidate, run the real test — `collidesWith`, or a circle overlap, or a line-of-sight trace.

`HomingProjectile` is the canonical example. Its broad phase is `nearby(quadtree, range)` where the range is the larger of the two summed half-extents, guaranteeing that a genuinely overlapping target is inside the window. Its narrow phase is `target in candidates && collidesWith(target)`. `DirectionalProjectile` does the same over a wider default window — three times its own width — and filters the candidates to `Alive`s before testing overlap.

Two rules follow. First, size the broad-phase window from the largest possible interaction distance, and be generous; a window that is too small produces silently missed collisions, while one that is too large only costs a few extra narrow-phase tests. Second, remember what the candidate list contains: every `VisibleObject` the world indexed, including deactivated ones, including objects on your own side, and including the querying object itself. Filter for what you actually want before doing anything expensive.

5.6 Limits and traps

The tombstone-reuse quirk. When `remove` empties a node it leaves the node in place, with its original coordinates, as a reusable slot. A later `insert` that passes through that node takes the slot — but the node keeps its *old* coordinates, so the newly inserted element is thereafter indexed at the position of the element that used to be there. Subsequent queries will find it in the wrong place. This is invisible to anyone using the per-frame rebuild idiom, because `clear()` discards the whole structure and `remove` is never called. If you maintain a tree incrementally, do not use `remove`; rebuild instead.

There is no radius query and no nearest-neighbour query. `retrieveBox` is the only way in. A circular query is a box query followed by a distance filter; a nearest-neighbour search is a box query with a widening window.

Nothing is bounded. The tree has no extent and no capacity, and coordinates are not validated. A NaN coordinate compares false against everything, so it lands in a predictable but useless quadrant and will generally never be found again.

Only top-level visible objects are in the world's tree. Sub-objects are not inserted, bare `GameObject`s are not inserted, and inactive objects *are*. If you need any other membership rule, build and maintain your own tree — the class is public and has no dependency on the rest of the engine.

Events

Eight facts the simulation reports about itself, and the small synchronous bus that delivers them in a defined order without ever letting a bad listener break a tick.

6.1 Why a bus

6.2 EventBus semantics

6.3 The GameEvent catalogue

6.4 Subscribing safely

6.5 Patterns

6.6 Anti-patterns

QUICK REFERENCE

EVENTBUS · GAMEEVENT

```
val sub = world.events.subscribe { event ->
    when (event) {
        is GameEvent.EntityDied ->
            board.credit(event.killer, event.entity)
        is GameEvent.EntitySpawned ->
            log.info("spawned {}",
                event.entity.entityId)
        else -> {}
    }
}

sub.cancel() // when your system goes away
```

EVENTBUS

MEMBER	BEHAVIOUR
subscribe(listener)	Registers it for everything published from now on. Returns a <code>Subscription</code> .
Subscription.cancel()	Stops delivery. Idempotent.
publish(event)	Delivers to every listener, in subscription order, on the calling thread.

FOUR GUARANTEES

- **Synchronous.** Delivery finishes before `publish` returns.
- **Ordered.** Listeners are called in subscription order; events in publication order.
- **Fault-isolated.** A listener that throws is caught and logged; the rest still run.
- **Flat.** Publishing from inside a listener queues rather than recursing.

NOT A CONCURRENCY PRIMITIVE

The bus does no synchronisation. It is delivered on whichever thread called `publish` — for built-in events, the tick thread. Do not subscribe from another thread while a tick is running, and do not block inside a listener.

SEALED, FOR NOW

`GameEvent` is a `sealed` interface in the library's module, so your own event types cannot join it. Exhaustive `when` is the upside; you will need your own bus for game-specific events.

THE GAMEEVENT CATALOGUE

EVENT	PAYLOAD	PUBLISHED WHEN
EntitySpawned	entity	A world takes ownership — in <code>add()</code> , or in tick phase 3 for an object appended directly. Fires again if an instance leaves and returns.
EntityRemoved	entity	Tick phase 5, as the object is dropped. After this, <code>byId</code> no longer resolves it.
AttackIssued	attacker, target	Synchronously inside <code>issueAttack</code> .
AttackLanded	attacker, target, damage	A swing connected <i>and was not evaded</i> , after the damage was applied.
DamageDealt	source?, target, amount	Health was removed through the combat path. <code>source</code> is null when nothing tracked it.
EntityDied	entity, killer?	Health reached zero and the death response was applied. Once per death; a resawner can fire it again.
AttackCancelled	attacker, formerTarget?	<code>cancelAttack()</code> was called on an attacking actor.
AttackEnded	attacker, formerTarget?, reason	The attack stopped by itself: <code>TARGET_DIED</code> or <code>TARGET_REMOVED</code> .
<i>Every combat event requires the attacker to be in a world — they are published through <code>world?.events</code>. A worldless <code>Alive</code> fights silently.</i>		

IN DETAIL

§6.1 – §6.6

6.1 Why a bus

Without an event stream, every system that cares about a kill has to be wired into the code that causes one. The scoreboard reaches into `Alive.die`; the kill feed reaches in next to it; the netcode adds a third call; the achievement system a fourth. Each addition edits a method that has nothing to do with scoring, and none of them can be tested without a full combat setup.

The bus inverts that. `Alive` announces what happened and knows nothing about who is listening; the scoreboard subscribes and knows nothing about how a kill is computed. The roadmap leans on this heavily — the delta netcode, the client event feed and the threat table are all planned as consumers of the same stream — but the immediate benefit is that game logic written on top of `GameTools` does not have to subclass anything to observe the simulation.

The set is deliberately small. Eight events cover entity lifecycle and combat; abilities, items and movement events are explicitly deferred to later phases rather than guessed at now.

6.2 EventBus semantics

The bus is about ninety lines and every one of its behaviours is a deliberate choice worth knowing.

Delivery is synchronous. `publish` walks the listener list on the calling thread and returns only when every listener has been called. There is no queue thread and no deferral. For built-in events the calling thread is whichever thread is running `World.tick()`, which means a listener's work happens *inside* the tick and its cost is part of the frame budget.

Order is defined twice over. Listeners are called in subscription order, and events are delivered in publication order. Both matter for determinism: a replay of the same run produces the same sequence of listener calls.

A throwing listener cannot break the tick. Each call is wrapped, and a failure is logged at `warn` with the event that caused it before delivery continues to the next listener. This is not permission to throw for control flow; it is a guarantee that a bug in a scoreboard does not corrupt a simulation.

Re-entrant publishing does not recurse. If a listener publishes an event while a delivery pass is running, the new event is appended to an internal queue and delivered after the current one finishes. The stack stays flat no matter how many events cascade, and the resulting order is breadth-like and deterministic. The consequence to be aware of is that a listener's own event is *not* delivered before the listener returns.

The listener list is copied before each pass. Subscribing or cancelling from inside a listener is therefore safe, and takes effect from the next event rather than the current one.

6.3 The GameEvent catalogue

The quick-reference table above lists all eight; what follows is what each one is actually good for and where it will surprise you.

EntitySpawned and EntityRemoved bracket an object's membership of a world. Spawned fires from `add()` immediately, or from tick phase 3 for an object appended straight to the list. Only *top-level* objects are announced — sub-objects are indexed and addressable but are considered part of their parent, not independent arrivals. Removed fires in phase 5, after which `byId` returns `null`. Together they are the natural driver for a network layer that maintains a client-side object table.

AttackIssued is the order, not the swing. It fires once, when `issueAttack` is called, on whatever thread called it — which for a command handler may not be the tick thread. It is the right hook for aggro: a unit that is targeted learns

about it immediately, before any damage.

AttackLanded reports a swing that connected. Read §4.8 before relying on it: despite what its KDoc says, the implementation publishes it only for swings that were *not* evaded, and only after the damage has already been applied. If you want a “swing attempted” signal that includes misses, use the **onAttack** hook instead.

DamageDealt carries the amount actually applied, after mitigation, and a nullable source. It is the correct event for a damage meter — but only for damage that went through the combat path. Projectile damage and direct **health.current** edits do not produce it (§3.7, §4.8).

EntityDied carries the victim and the actor whose damage last reduced its health, which is the library's whole notion of kill credit. It fires once per death, after the death response has been applied — so by the time you see it, a **REMOVAL** victim is already queued for removal and a **RESPAWN** victim is already back at full health at its spawn point. A respawning unit fires the event again each time it dies.

AttackCancelled and **AttackEnded** are the two exits from the attack loop, kept separate so that a listener can tell a player's decision from the simulation's. UI that shows “attacking X” should clear on either.

6.4 Subscribing safely

subscribe takes a **fun interface**, so a lambda works directly, and returns a **Subscription** whose **cancel** is idempotent. Hold that handle: the bus holds a strong reference to every listener for the life of the world, so a listener that is never cancelled is never collected, and a listener that closes over a large object keeps it alive too. Systems with a lifecycle shorter than the world's — a UI panel, a round-scoped scoreboard — must cancel.

There is one filtering mechanism, and it is **when**. Every listener receives every event, so the first line of a listener is nearly always a type check. Keep the check cheap and the body short: the work happens inside the tick, and a listener that takes a millisecond costs two per cent of a 20 Hz frame.

Above all, do not mutate the world from inside a listener without thinking about the tick phase you are in. Adding an object during phase 4 is safe — the pass iterates a copy — but the newcomer is not numbered or ticked until the next frame. Removing is safest through **removeList**, which is drained in phase 5. Anything that must happen “after this frame” should be recorded by the listener and acted on by your own code after **tick()** returns.

6.5 Patterns

Kill feed and scoring. Subscribe once at world construction, match on `EntityDied`, credit `event.killer` when it is non-null and record an environmental death otherwise. §10.5 has the code.

Client object table. Match on `EntitySpawned` and `EntityRemoved` and accumulate the ids into a pair of sets, then send those alongside the world-state broadcast so a client can create and destroy its local representations without diffing the whole snapshot list. This is the first step towards the delta protocol on the roadmap.

Threat and aggro. Accumulate `DamageDealt` amounts per `(target, source)` pair into a table, and have your AI pick the highest-threat attacker when choosing what to swing at. `AttackIssued` gives you an initial threat spike before any damage has been dealt.

Observability. A listener that logs every event at `debug`, with the tick number, is the fastest way to understand an unfamiliar simulation — and the fastest way to diff two runs when chasing a determinism bug (§2.7).

6.6 Anti-patterns

Doing real work in a listener. Path-finding, file I/O, a database write or a network round-trip inside a listener happens inside the tick and stalls the whole simulation. Record the fact; do the work afterwards.

Using the bus as a command channel. Events describe what has already happened. Publishing a synthetic event to make something happen inverts the dependency the bus exists to remove, and the queueing behaviour means your “command” will not have run by the time `publish` returns.

Assuming an event means the object is still there. By the time `EntityRemoved` is delivered the object is out of the world and unresolvable by id. Capture what you need from the event's payload rather than looking it up again.

Subscribing per entity. One listener that switches on the event is cheaper and easier to reason about than one listener per object, and it does not leak when objects die.

Network

One UDP socket, a three-token handshake, two verbs, and a set of threading rules that decide whether your server survives contact with players.

7.1 The transport underneath

7.2 The handshake

7.3 Admission, capacity and reconnection

7.4 Message routing

7.5 Broadcasting world state

7.6 Targeted sends and disconnection

7.7 Keepalives and NAT

7.8 Threading and lifecycle

7.9 `MouseAction`

7.10 Failure modes

QUICK REFERENCE

PROTOCOL · API · THREADING

THE WIRE PROTOCOL, IN FULL

DIR.	DATAGRAM	MEANING
C → S	Iam <name>	Handshake. A trailing address token is accepted and ignored. Handled entirely by WebTools.
S → C	REGISTERED	The bare token, sent from the common socket straight back to the datagram's source address and port.
C → S	KA	Keepalive. Consumed silently by WebTools; never reaches your callbacks. Send one every ~20 s.
C → S	INPUT <json>	A serialized <code>MouseAction</code> . Decoded and delivered to <code>onPlayerInput</code> . A malformed body is logged and dropped.
C → S	<anything else>	Passed verbatim, trimmed, to <code>onPlayerMessage</code> . This is where your own verbs live.
S → C	STATE <json>	A JSON array of polymorphic <code>DrawableSnapshot</code> s. Sent by <code>broadcast</code> .

Everything travels over one shared UDP socket — `MultiConnectionUDPServer.COMMON_LISTEN_PORT`. There is no per-player port pair.

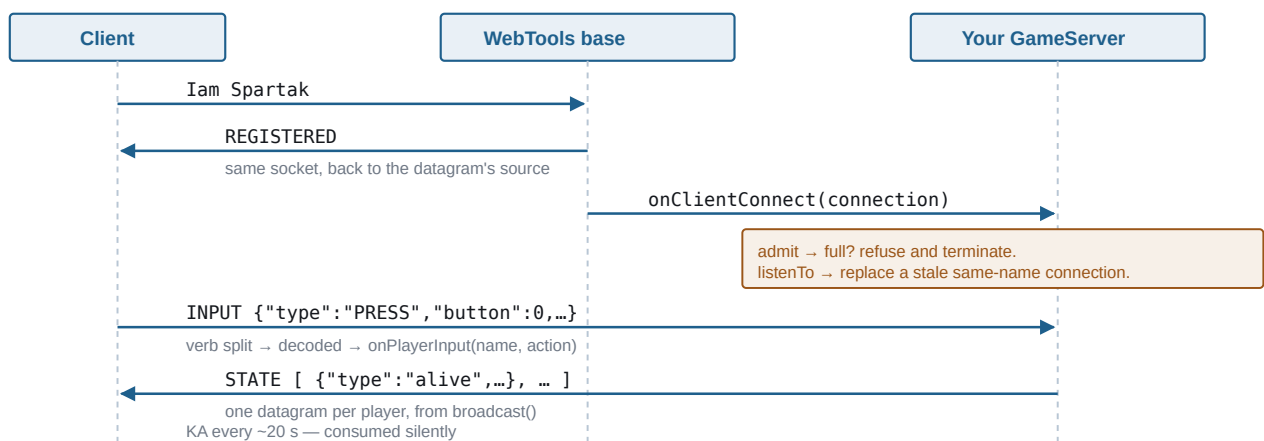


FIGURE 7.1 The base class owns the handshake; `GameServer` owns admission, routing and broadcast.

GAMESERVER

MEMBER	BEHAVIOUR
<code>GameServer(maxConnections, onPlayerMessage, onPlayerInput)</code>	Constructing <i>starts</i> the server. Both callbacks default to no-ops.
<code>playerCount: Int</code>	How many are connected right now.
<code>playerNames: Set<String></code>	A detached copy of the current names.
<code>broadcast(Iterable<VisibleObject>)</code>	Snapshot the visible ones and send. <code>Result</code> .
<code>broadcast(List<DrawableSnapshot>)</code>	Send snapshots you already took. <code>Result</code> .
<code>pushToAllPlayers(message)</code>	Raw send to every tracked player. Prefer over the inherited <code>pushToAll</code> .
<code>push(playerName, message)</code>	Raw send to one. Fails if no such player.
<code>disconnect(playerName)</code>	Unbind their handler. They may handshake again.
<code>shutDown()</code>	Forget every player, then <code>stop()</code> . No restart afterwards.
<code>STATE_VERB / INPUT_VERB</code>	<code>"STATE"</code> / <code>"INPUT"</code> .
<code>inputMessage(action)</code>	Builds the datagram a <i>client</i> sends. Use it in your client code.

THREADING — READ THIS BEFORE WRITING A CALLBACK

`onPlayerInput` and `onPlayerMessage` run on WebTools' per-player listener threads, *not* the tick thread, and run concurrently for different players. Touching the world from inside one is a data race. Queue the input; drain the queue at the top of your tick.

ONE INSTANCE PER JVM

The common port is fixed. Constructing a second `GameServer` before `shutDown()` ing the first fails to bind. This is also why the build serialises every test task that starts one.

MOUSEACTION

`type` ∈ {MOVE, PRESS, RELEASE} · `button` a GLFW code, or `-1` for MOVE · `x`, `y` in *window pixels*, origin top-left. Not world coordinates — you map them.

7.1 The transport underneath

`GameServer` is a subclass of WebTools' `MultiConnectionUDPServer`, and the division of labour between the two is worth being precise about, because half of what you will observe on the wire is the base class's behaviour rather than GameTools'.

The base class owns the socket. It binds one fixed UDP port — `MultiConnectionUDPServer.COMMON_LISTEN_PORT` — spawns a thread that listens for handshakes on it, replies to them, creates a `Connection` per accepted client, and multiplexes every subsequent datagram for every player over that same socket. It also consumes keepalives silently. What it does *not* know is anything about a game: it has no capacity limit, no message vocabulary and no notion of world state.

That is `GameServer`'s half. It caps the player count, keeps a table of who is connected, splits each incoming datagram into a verb and a payload, decodes the one verb it understands, serialises world state on the way out, and exposes targeted and broadcast sends.

Because the wire behaviour is jointly owned, the WebTools version is part of GameTools' observable protocol rather than an implementation detail — the 3.0.0 release was a breaking change precisely because WebTools 2.0.0c replaced a per-player port pair with the shared socket and changed the handshake reply. Treat a WebTools upgrade as a protocol change.

7.2 The handshake

A client sends `Iam <name>` to the common port. A trailing address token is tolerated and ignored, a leftover from an earlier protocol version. The server replies with the bare token `REGISTERED`, sent from that same common socket straight back to the source address and port of the datagram it received — which is what makes the whole thing traverse NAT: the reply arrives on the mapping the client's own datagram just created.

The client must therefore read the reply, and every subsequent broadcast, on the same socket it sent `Iam` from. A client that sends its handshake from an ephemeral socket and then listens on a different one will hear nothing.

Once the reply is out, the base class constructs a `Connection` and calls `onClientConnect`. Note the ordering: the client has *already been told it is registered* before your code gets any say. A refusal at this point cannot be a handshake rejection, which shapes everything in the next section.

7.3 Admission, capacity and reconnection

`onClientConnect` runs a two-step pipeline — decide whether to admit, then start listening — and funnels both failure paths into one place that logs the reason, removes the connection from the player table by value, and terminates it.

Admission refuses in two cases. The first is capacity: if the table already holds `maxConnections` players *and* this name is not one of them, the connection is refused. The second is a startup race. `MultiConnectionUDPServer` begins accepting handshakes from its own `init` block, which the JVM runs before a subclass's fields are initialised — so a client that is already retrying `Iam` when the process boots can reach `onClientConnect` while the player table is still `null`. The class detects that window with a deliberately odd null check and refuses cleanly instead of failing on the handshake thread.

Reconnection is handled by name. A player handshaking under a name that is already connected is always admitted — they do not count a second time against the cap — and their previous connection is terminated so it stops holding a message handler. This is what makes a client recover from a changed NAT mapping or a restarted app without an explicit disconnect.

A REFUSED CLIENT IS NOT TOLD

Because `REGISTERED` has already gone out, refusing means terminating the connection silently. The client believes it is connected; nothing on the server ever answers it again. If your game needs a “server full” screen, send an explicit message before terminating — or, better, have clients treat a lack of `STATE` within a timeout as a failure to join.

Names are the identity for everything: routing, targeted sends, disconnection. The server does not authenticate them, and any client may claim any name — including a name already in use, which as described above kicks the incumbent. Anything resembling a real deployment needs an authentication layer in front of this; the roadmap places one behind an `AuthProvider` seam in a future `gametools-session` module.

7.4 Message routing

Every datagram from an admitted player arrives at one dispatcher. It trims the text, splits it once on the first space into a verb and a remainder, and switches on the verb. Exactly one verb is recognised: `INPUT`, whose remainder is decoded as a `MouseAction` and handed to `onPlayerInput` along with the sender's name. A body that will not parse is logged at `warn` and dropped — it reaches neither callback. Everything else, whole and trimmed, goes to `onPlayerMessage`.

That catch-all is the extension point for your own protocol. Define your own verbs — `ORDER`, `CHAT`, `PING` — parse them in `onPlayerMessage`, and you have a complete inbound vocabulary without touching the library. Keep the first token distinct from `INPUT`, `KA` and `Iam`.

On the client side, `GameServer.inputMessage(action)` builds the exact `INPUT <json>` datagram the server expects, so both ends agree on the encoding without duplicating it.

7.5 Broadcasting world state

There are two broadcast overloads. The convenient one takes an `Iterable<VisibleObject>`, filters out anything whose `visible` flag is false, snapshots each remaining object as the most specific kind that fits it, and sends. The explicit one takes a list of snapshots you have already built, which is what you want when the same snapshots feed something else — a recorder, a spectator view — or when you are filtering per player.

Either way the payload is a single JSON array, prefixed with the `STATE` verb and a space, pushed to every tracked player. The array serialises polymorphically: each element carries a `type` discriminator, so a mixed list of alive, actor and plain visible snapshots decodes correctly on the far end (§8.2).

Two properties of this design set its scaling limit. It is a *full-state* broadcast — every visible object, every tick, to everyone — and it is *JSON*. A hundred objects of moderate complexity is comfortably a few tens of kilobytes per tick per player, which at 20 Hz is already a real bandwidth figure, and a UDP datagram that exceeds the path MTU will fragment or be dropped outright. Both the delta protocol and the binary codec that fix this are on the roadmap; until then, keep the broadcast list small deliberately — cull by distance, by team vision, or by only sending objects that changed — and consider broadcasting at a lower rate than you tick.

WHERE TO CALL IT

Broadcasting reads the world, so it must happen on the tick thread. If you are using `SimulationLoop`, do it in the `onTick` callback; if you drive the loop yourself, do it immediately after `world.tick()` returns.

7.6 Targeted sends and disconnection

`push(playerName, message)` sends one message to one player, addressed to that player's own post-NAT peer over the shared socket, and fails with a `NoSuchElementException` if the name is not connected.

`pushToAllPlayers(message)` sends to everyone the server currently tracks; it pushes to every player regardless of whether earlier ones failed, and reports the first failure it saw.

Prefer `pushToAllPlayers` over the inherited `pushToAll`. The inherited method reaches every connection WebTools has ever registered for this instance — including the handshake this class refused for being over capacity, and the stale connection of a player who has since reconnected from a new address — because WebTools never prunes a registration once a handshake completes. `pushToAllPlayers` means “every player I currently track”, which is almost always what you meant.

`disconnect(playerName)` removes the player from the table and terminates their connection, unbinding its handler. They are free to handshake again afterwards. `shutDown()` clears the table and calls the inherited `stop()`, which tears down every connection and releases the common port. There is no restart: discard the instance.

7.7 Keepalives and NAT

Because every player's traffic rides one shared socket, and because the server addresses each player at the post-NAT address its handshake arrived from, the whole scheme depends on the client's NAT mapping staying alive. Most consumer NATs expire a UDP mapping after thirty to sixty seconds of silence, and a client that only listens — which is exactly what a player does while watching the game between clicks — sends nothing.

The contract is therefore that the *client* sends a bare `KA` datagram on the socket it handshook from, roughly every twenty seconds. WebTools consumes it silently: it never reaches `onPlayerMessage` or `onPlayerInput`, and it produces no reply. Nothing in the server enforces this, so a client that omits keepalives will simply stop receiving broadcasts at some unpredictable point, usually while its player is standing still. It is the single most common cause of “the game froze for one player”.

7.8 Threading and lifecycle

Construction starts the server. The base class spawns its handshake thread from its own `init` block, so players may begin connecting as soon as your constructor returns — and, as §7.3 describes, very slightly before it does. Give the constructor its final callbacks; do not plan to install them afterwards.

Both callbacks run on WebTools' listener threads, one per player. Two consequences follow. They may run concurrently with each other, so anything they touch must be thread-safe. And they run concurrently with your tick

thread, so touching the world from inside one races against the simulation. The server's own player table is a `ConcurrentHashMap` for exactly this reason; your game state needs the same treatment or, far better, the queue-and-drain pattern in §10.6.

Callbacks should also return quickly. A slow callback blocks that player's listener thread and therefore delays every subsequent datagram from that player.

7.9 MouseAction

`MouseAction` is the one structured inbound type: a serializable data class with a `type` of `MOVE`, `PRESS` or `RELEASE`, a `button`, and an `x/y` pair. The button is a GLFW mouse-button code — zero is the left button — and is `-1` for a `MOVE`, where no button is involved.

The coordinates are the important part: they are **window pixels, with the origin at the top-left**, exactly as the client's windowing system reported them. They are not world coordinates, and the server has no way to convert them on its own, because it knows nothing about the client's window size, camera position or zoom. Mapping pixels to world space is your game's job, and it needs information the client must send you — typically a camera origin and scale in a message of your own.

This also means `MouseAction` is a low-level input record rather than a command. The roadmap replaces it with a generic action map — named, bindable actions with analog and digital values, of which mouse and keyboard are merely sources — but the current shape is deliberately raw, and building your own command verbs on top of `onPlayerMessage` is the sanctioned way to go beyond it today.

7.10 Failure modes

UDP is unreliable and the library adds nothing. There is no acknowledgement, no retransmission, no ordering and no duplicate suppression. A dropped `STATE` datagram is simply a frame the client never saw; because each broadcast is a full state, the next one repairs it, which is why full-state broadcast is a reasonable starting point. A dropped `INPUT`, by contrast, is a click that never happened.

Large payloads fragment. A JSON world state larger than the path MTU is fragmented by IP, and the loss of any one fragment loses the whole datagram. Keep the broadcast under roughly a kilobyte and a half if you can, and expect increasing loss as it grows.

A malformed `INPUT` disappears. It is logged at `warn` and dropped, deliberately, so a hostile or buggy client cannot crash the server with bad JSON. If you are debugging a client that “sends input the server ignores”, look in the log first.

Serialisation failure is reported, not thrown. If a snapshot list cannot be encoded, `broadcast` logs at `error` and returns a failed `Result`. Check it — a silently discarded `Result` here means a frame nobody received.

There is no disconnection detection. Nothing times a player out. A client that vanishes stays in the player table, keeps counting against `maxConnections`, and keeps receiving datagrams into the void until something calls `disconnect`. If your game needs presence, build it: have clients send a heartbeat verb of your own through `onPlayerMessage`, record the arrival time, and disconnect the silent.

Serialization

Nine immutable wire types, one polymorphic family, and the rules that decide what a client is told and what it never hears about.

- 8.1 The snapshot pattern
- 8.2 The drawable family
- 8.3 Field-by-field wire reference
- 8.4 Sub-object trees and visibility

- 8.5 Identity on the wire
- 8.6 Compatibility and evolution
- 8.7 Geometry snapshots

QUICK REFERENCE

TYPES · JSON · COMPATIBILITY

EVERY SNAPSHOT TYPE

TYPE	CAPTURES
PointSnapshot	<code>x</code> , <code>y</code>
DimensionsSnapshot	<code>width</code> , <code>height</code>
TwoDoublesSnapshot	<code>first</code> , <code>second</code> — any <code>TwoDoubles</code>
ColorSnapshot	<code>r</code> , <code>g</code> , <code>b</code> , <code>a</code>
CombinedStatSnapshot	<code>value</code> , <code>maxValue</code>
BuffSnapshot	<code>name</code> , <code>durationTicks</code> , suppressed capability ids
GameObjectSnapshot	<code>location</code> , <code>buffs</code>
VisibleObjectSnapshot	drawable state + nested sub-objects
ActorSnapshot	the above + <code>speed</code> , <code>destination</code>
AliveSnapshot	the above + health, faction, owner, five combat stats

All are `@Serializable` data classes built through an *infix companion* *from*.

COMPOSITION, NOT FLATTENING

An `AliveSnapshot` contains an `ActorSnapshot`, which contains a `VisibleObjectSnapshot`, which contains a `GameObjectSnapshot`. The JSON nests to match, so a position lives at `actor.visibleObject.gameObject.location`.

EFFECTIVE VALUES ONLY

Combat stats cross the wire as the single `Double` a `ModularStat` currently computes. The modifiers behind it are never sent — a client sees *13.0 speed*, not *10 base +30 %*.

DEFAULTS ARE NOT ENCODED

`Json`'s default configuration omits any field equal to its declared default. In practice that means `id` is absent for an unowned object and `buffs` is absent when empty. Clients must treat both as optional.

NOT ON THE WIRE

NEVER SENT

active

visible

movement

attack state / progress

respawn / deathResponse

world / owner object

every StatMod

capabilities (the set)

tickCount / seed

Invisible objects are filtered out entirely rather than sent with a flag.

// One STATE broadcast, formatted for reading.

```
[
  {
    "type": "alive",
    "id": 1,
    "actor": {
      "id": 1,
      "visibleObject": {
        "id": 1,
        "gameObject": {
          "location": { "x": 148.2, "y": 131.6 },
          "buffs": [
            { "name": "haste", "durationTicks": 38,
              "suppressedCapabilities": [] }
          ]
        }
      },
      "dimensions": { "width": 40.0, "height": 40.0 },
      "color": { "r": 255, "g": 255, "b": 255, "a": 255 },
      "texture": "default.png",
      "angle": 32,
      "turns": false,
      "subObjects": [
        { "type": "visibleObject", "id": 2,
          "gameObject": { "location": { "x": 148.2, "y": 107.6 }
        }
      ]
    },
    "dimensions": { "width": 30.0, "height": 8.0 },
    "color": { "r": 255, "g": 0, "b": 0, "a": 255 },
    "texture": "default.png", "angle": 0,
    "turns": false, "subObjects": [] }
  ],
  "speed": 13.0,
  "destination": { "x": 400.0, "y": 250.0 }
},
{
  "health": { "value": 187.5, "maxValue": 250.0 },
  "faction": "neutral",
  "ownerName": "Spartak",
  "damage": 12.5, "attackTime": 1.7,
  "attackSpeed": 100.0, "attackRange": 750.0,
  "evasion": 0.0
}
]
```

IN DETAIL

§8.1 – §8.7

8.1 The snapshot pattern

Not one live type in GameTools is `@Serializable`. Every networkable type instead has a separate, immutable companion whose name is the type plus `Snapshot`, and the only way to make one is an `infix` function called `from` on that companion's companion object: `AliveSnapshot` from `hero`, `PointSnapshot` from `location`.

The separation buys three things. It makes the instant of capture explicit — a snapshot is a value taken at a known moment, not a live view that might change while a serializer is walking it. It keeps the wire format from being hostage to the simulation's internals, so `Alive` can gain a field without that field automatically appearing in every client's parser. And it means the mutable objects at the heart of the tick — `Point`, `Dimensions`, the stats — never need to be safe for concurrent reading by a serializer.

The cost is a hand-written mapping per type, which is why the family is small and why each `from` function is a handful of lines. When you add a field to an entity and want clients to see it, you edit two places: the snapshot's constructor and its `from`.

8.2 The drawable family

`DrawableSnapshot` is a sealed interface with three implementations and two members: an `id` and a list of sub-object snapshots. Its companion carries the factory, `DrawableSnapshot from visibleObject`, which picks the most specific kind that fits — `AliveSnapshot` for an `Alive`, otherwise `ActorSnapshot` for an `Actor`, otherwise a plain `VisibleObjectSnapshot`. A `Projectile`, being an `Actor`, travels as an `ActorSnapshot`; its damage value is not sent.

Because the interface is sealed and each implementation carries a `@SerialName`, `kotlinx.serialization` encodes the family polymorphically: every object in a broadcast array carries a `type` field holding `"alive"`, `"actor"` or `"visibleObject"`, and one mixed list decodes back into the right types on the far end without the client having to guess from which fields are present.

The three form a chain by *composition*, not by inheritance of fields. An `AliveSnapshot` has an `actor` property holding an `ActorSnapshot`, which has a `visibleObject` property holding a `VisibleObjectSnapshot`, which has a `gameObject` property holding a `GameObjectSnapshot`. The JSON nests to match, and a client reading a unit's position walks `actor.visibleObject.gameObject.location`. Deeper than it would be if the fields were flattened, but it means each level's mapping is written once and reused by the level above.

One consequence of the composition is worth knowing before you write a parser: on `ActorSnapshot` and `AliveSnapshot`, `subObjects` is a computed property that reads through to the nested visible-object snapshot, not a constructor parameter. Computed properties are not serialized, so those two types have no `subObjects` field in their JSON at all — the children live at `...visibleObject.subObjects`. Only `VisibleObjectSnapshot` carries the array directly.

8.3 Field-by-field wire reference

VISIBLEOBJECTSNAPSHOT — "VISIBLEOBJECT"

FIELD	JSON TYPE	SOURCE
id	number	<code>entityId.raw</code> ; omitted when <code>0</code>
gameObject	object	location and buffs
dimensions	object	<code>width</code> , <code>height</code>
color	object	<code>r g b a</code> , each 0–255
texture	string	the texture name
angle	number	whole degrees, 0–359
turns	boolean	is <code>angle</code> meaningful
subObjects	array	visible children, recursively

ACTORSNAPSHOT — "ACTOR"

FIELD	JSON TYPE	SOURCE
id	number	as above
visibleObject	object	the whole drawable layer
speed	number	<code>speed.value</code> — effective
destination	object	the point being moved towards

GAMEOBJECTSNAPSHOT & BUFFSNAPSHOT

FIELD	SOURCE
location	<code>x</code> , <code>y</code>
buffs	array; omitted when empty
buffs[].name	the buff's name
buffs[].durationTicks	ticks left; negative = indefinite
buffs[].suppressedCapabilities	array of capability <code>id</code> strings

ALIVESNAPSHOT — "ALIVE"

FIELD	JSON TYPE	SOURCE	NOTES
id	number	<code>entityId.raw</code>	Omitted when <code>0</code> . Repeated at every nested level.
actor	object	the whole movement layer	And, through it, the drawable and base layers.
health	object	<code>value</code> + <code>maxValue</code>	The maximum is the <i>modified</i> ceiling.
faction	string	<code>faction</code>	"neutral" unless you set it.
ownerName	string?	<code>owner?.name</code>	The name only — never the <code>Player</code> object.
damage	number	<code>damage.value</code>	All five are effective values with every <code>StatMod</code> already folded in. A client cannot see the modifiers.
attackTime	number	<code>attackTime.value</code>	
attackSpeed	number	<code>attackSpeed.value</code>	
attackRange	number	<code>attackRange.value</code>	
evasion	number	<code>evasion.value</code>	

8.4 Sub-object trees and visibility

Sub-objects are snapshotted recursively, in list order, and each is snapshotted as its own most-specific kind — a sub-object that happens to be an `Actor` becomes an `ActorSnapshot`. The recursion filters on the child's own

`visible` flag, so hiding a health bar removes it from the payload without affecting its parent.

Visibility is filtered in two places and it is worth being clear about which does what. The convenience `broadcast(Iterable<VisibleObject>)` overload drops invisible *top-level* objects before snapshotting; the recursive snapshot drops invisible *children*. If you build your snapshot list yourself and pass it to the explicit `broadcast(List<DrawableSnapshot>)` overload, only the second filter is applied — you own the first.

Remember that `visible` tracks `active`: deactivating an object hides it too. That is usually what you want — a spent projectile should stop appearing — but it also means an object you deactivate for simulation reasons silently vanishes from clients. Set `visible` back to `true` afterwards if you want an inert object still drawn.

8.5 Identity on the wire

Every `DrawableSnapshot` carries the object's `EntityId.raw` as a plain number, which is what lets a client keep a stable table of objects across frames instead of re-deriving everything from list position each tick. The id appears at each nested level of the same object — the outer `id`, the `actor.id` and the `actor.visibleObject.id` are all the same value — which is redundant but harmless, and a consequence of each layer's snapshot being independently constructible.

Sub-objects carry their own ids, assigned by the same world enrolment, so a client can address a health bar distinctly from the unit that owns it.

The value `0` — `DrawableSnapshot.UNIDENTIFIED`, matching `EntityId.UNASSIGNED` — means “not addressable”. It appears for an object that no world has numbered, and it is also what a client sees when decoding a pre-3.1 payload that had no id field at all. Collapsing those two cases into one value is deliberate: a client can handle both with a single check.

8.6 Compatibility and evolution

Two mechanisms keep old and new ends talking. Fields with declared defaults — `id` and `buffs` today — decode successfully when absent, which is how a 3.1 client reads a 3.0 payload. And because kotlin's default configuration does not encode a value equal to its default, those same fields are omitted on the way out when they are at their default, which keeps the payload smaller and means a client must treat them as optional even from a current server.

Fields *without* defaults are mandatory in both directions: adding one to a snapshot type breaks every client that has not been updated. The rule for evolving the format is therefore simple — **new fields always get a default** — and the rule for removing one is that you cannot, without a version bump, because an old client will fail to find it.

The other axis of change is the polymorphic discriminator. The three `@SerialName` values are part of the protocol; renaming one is a breaking change even though nothing in Kotlin refers to the string. Adding a fourth implementation to the sealed family is safe for servers and breaking for clients, which will encounter a `type` they do not know.

Nothing in the payload identifies its protocol version. If your deployment mixes versions, send one yourself — a verb of your own through `onPlayerMessage` at handshake time is the natural place.

8.7 Geometry snapshots

Three small types live in `com.spartanlabs.geometry.serializations` rather than with the game objects, because they belong to GeneralTools' geometry rather than to GameTools' entities. `PointSnapshot` captures a point's two coordinates, `DimensionsSnapshot` a width and height, and `TwoDoublesSnapshot` any `TwoDoubles` at all as a plain `first/second` pair.

All three follow the same `infix from` pattern and are usable on their own, which is the point: if your game sends geometry alongside world state — a camera origin, a selection rectangle, a waypoint list — these are the encodings the rest of the protocol already uses, and reusing them keeps one convention on the wire instead of two.

API Reference

Every public declaration in the library, by package. A lookup table, not a read — the behaviour behind each entry is in the chapter it belongs to.

9.1 gaming.gameobjects — entities
9.2 gaming.gameobjects — stats & buffs
9.3 gaming.event
9.4 gaming.simulation

9.5 gaming.spatial
9.6 gaming.networking
9.7 geometry.serializations & operators

9.1 com.spartanlabs.gaming.gameobjects — entities

DECLARATION	NOTES
abstract class GameObject	Root of every simulated entity. §3.1
GameObject(location: Point = Point())	Primary constructor.
GameObject(x: Double, y: Double)	Secondary.
val location: Point	Mutable point in a stable cell. Mutate, do not reassign.
var entityId: EntityId	<code>internal set</code> . <code>UNASSIGNED</code> until a world numbers it.
open var active: Boolean = true	While false, <code>tick()</code> is a no-op.
fun tick()	Age buffs → <code>onUpdate()</code> → prune. Not open.
protected open fun onUpdate()	The override point. Call <code>super</code> first.
infix fun distanceFrom(other): Result<Double>	Fails on a NaN coordinate.
open val capabilities: Set<Capability>	Empty. Extend as <code>super.capabilities + ...</code> .
fun can(capability): Boolean	In the set and not suppressed by any active buff.
open val stats: Map<String, Moddable>	Empty. Declare as a getter.
val buffs: List<Buff>	Read-only copy, in application order.
fun applyBuff(buff) / removeBuff(buff)	Attach / detach; both fire the buff's hooks.
fun dispel(name: String): Int	Remove every buff of that name; returns the count.
open class VisibleObject : GameObject	Adds drawable state. §3.2
VisibleObject(area, color, texture)	Primary. Defaults 25×25, opaque white, <code>"default.png"</code> .
VisibleObject(dimensions, location)	Four secondary convenience constructors.
VisibleObject(width, height, location)	
VisibleObject(dimensions, x, y)	
VisibleObject(width, height, x, y)	
val area: Square	Its location is the object's <code>location</code> .
var dimensions: Dimensions	Reads and writes through to <code>area</code> .
var color: Color · var texture: String	Rendering state.
var angle: Int	Whole degrees CCW; normalised to 0–359 on assignment.
var turns: Boolean = false	Whether <code>angle</code> is meaningful.
var visible: Boolean = true	Whether it is broadcast. Follows <code>active</code> .
override var active: Boolean	Also assigns <code>visible</code> .
val subObjects: MutableList<VisibleObject>	Children, drawn in insertion order. Not ticked, not indexed spatially.

DECLARATION	NOTES
<code>open fun collidesWith(other): Boolean</code>	Centred AABB; touching counts. Override for other shapes.
open class Actor : VisibleObject	Adds motion. §3.3
<code>Actor(location: Point, dimensions: Dimensions)</code>	Both default.
<code>var speed: ModularStat = ModularStat(10.0)</code>	Units per tick. Buff key <code>"speed"</code> .
<code>var movement: Movement = Targeting</code>	Assigning clears the settled flag.
<code>var destination: Point</code>	Assigning re-aims <code>angle</code> and clears the settled flag.
<code>val isAtDestination: Boolean</code>	Exact point equality.
<code>val isOneStepAway: Result<Boolean></code>	Distance < <code>speed</code> .
<code>val locmod: Point</code>	This tick's displacement. NaN when already there.
<code>fun nearby(quadtree, range): List<VisibleObject></code>	Square broad-phase window; includes this actor.
<code>override val capabilities</code>	adds <code>CoreCapability.MOVE</code>
sealed class Movement	How an actor steps. §3.4
<code>data object Targeting</code>	Go there and stop. The default.
<code>data object Persistent</code>	Go there every tick, forever.
<code>data object Directional</code>	Step along <code>angle</code> ; ignore <code>destination</code> .
<code>data class Homing(val target: GameObject)</code>	Re-aim at the target each tick, then pursue.
open class Alive : Actor	Adds combat. §3.5, §4.7
<code>Alive(location, dimensions, maxHealth)</code>	<code>maxHealth</code> must be positive.
<code>var health: CombinedStat</code>	Buff key <code>"health"</code> .
<code>var damage: ModularStat = 10.0</code>	Five combat stats; buff keys <code>"damage"</code> , <code>"attackTime"</code> , <code>"attackSpeed"</code> , <code>"attackRange"</code> , <code>"evasion"</code> .
<code>var attackTime: ModularStat = 1.7</code>	
<code>var attackSpeed: ModularStat = 100.0</code>	
<code>var attackRange: ModularStat = 750.0</code>	
<code>var evasion: ModularStat = 0.0</code>	
<code>var world: World?</code>	Set by <code>World.add</code> . Needed for removal and events.
<code>var faction: String = "neutral"</code>	Never read by the library.
<code>var owner: Player?</code>	Two-way with the player's roster.
<code>val hasOwner: Boolean</code>	
<code>fun issueAttack(target: Alive)</code>	Begin the chase-and-swing loop.
<code>fun cancelAttack()</code>	End it from outside. Keeps the destination.
<code>enum class AttackEndReason</code>	<code>TARGET_DIED</code> , <code>TARGET_REMOVED</code> .
<code>val isAlive: Boolean</code>	<code>health.current > 0.0</code> .

DECLARATION	NOTES
<code>var respawn: Point</code>	Defaults to the creation position.
<code>var deathResponse: DeathResponse</code>	<code>REMOVAL</code> or <code>RESPAWN</code> .
<code>val healthBar: VisibleObject</code>	Red, 20 % of actor height, auto-maintained.
<code>protected open fun die() / onDeath()</code>	The death response, and the hook after it.
<code>protected open fun on...()</code>	Ten combat hooks. Full list in §4.10.
<code>const val DEFAULT_FACTION</code>	<code>"neutral"</code> .
<code>class Player(val name: String)</code>	Roster owner. §3.6
<code>val ownedAlives: List<Alive></code>	In acquisition order.
<code>val livingAlives: List<Alive></code>	Filtered on every read.
<code>fun own(alive): Boolean</code>	Transfers from a previous owner. False if already owned.
<code>fun disown(alive): Boolean</code>	False if this player does not own it.
<code>fun owns(alive): Boolean</code>	
<code>abstract class Projectile : Actor</code>	A travelling payload. §3.7
<code>Projectile(location, dimensions, damage)</code>	
<code>val damage: Double</code>	A plain number, not a stat.
<code>protected fun dealDamageTo(target: Alive)</code>	Subtracts directly. No evasion, no hooks, no events.
<code>class HomingProjectile : Projectile</code>	Chases one target, hits once, deactivates.
<code>HomingProjectile(location, dimensions, damage, target, quadtree)</code>	Installs <code>Movement.Homing</code> on itself.
<code>class DirectionalProjectile : Projectile</code>	Pierces a line for a fixed lifetime.
<code>DirectionalProjectile(location, dimensions, damage, directionAngle, maxDuration, quadtree, searchRadius = width × 3)</code>	<code>maxDuration</code> must be positive. Installs <code>Movement.Directionals</code> .
<code>class World(val seed: Long = Random.nextLong())</code>	The container. §2.1
<code>val rng: RandomSource</code>	Seeded from <code>seed</code> .
<code>var tickCount: Long</code>	<code>private set</code> .
<code>val gameObjects: ArrayList<GameObject></code>	Public and mutable.
<code>val removeList: ArrayList<GameObject></code>	Drained at the end of the tick.
<code>val quadtree: Quadtree<Double, VisibleObject></code>	Rebuilt every tick.
<code>val events: EventBus</code>	

DECLARATION	NOTES
<code>fun add(gameObject)</code>	Add + number + index + wire <code>Alive.world</code> + announce.
<code>fun byId(id): GameObject?</code>	<code>null</code> once the object has gone.
<code>fun tick()</code>	Five phases. §2.3
<code>value class EntityId(val raw: Long)</code>	Comparable, prints as <code>#n</code> . §2.4
<code>val UNASSIGNED: EntityId</code>	Raw <code>0</code> .

9.2 com.spartanlabs.gaming.gameobjects — stats, buffs, wire types

DECLARATION	NOTES
interface Moddable	What a buff can modify. §4.4
fun applyMod(mod) / removeMod(mod)	Removal matches on the mod's <i>name</i> .
class ModularStat(var base: Double = 0.0)	A modifiable number. §4.1
var value: Double	<code>internal set</code> ; recomputed eagerly.
operator plus / minus / times / div	Against <code>Double</code> and <code>ModularStat</code> .
operator compareTo · unaryPlus · unaryMinus	Ordering and sign, by <code>value</code> .
override fun applyMod / removeMod	
data class StatMod(name, value, type, stackingType)	One adjustment. §4.2
enum Type	<code>ADDITIVE</code> , <code>MULTIPLICATIVE</code> <code>DEFAULT</code>
enum StackingType	<code>NONE</code> , <code>SETTING</code> , <code>INCREMENTING</code> , <code>ADDITIVE</code> <code>DEFAULT</code> , <code>MULTIPLICATIVE</code>
data class CombinedStat	Current against a modifiable ceiling. §4.3
CombinedStat(startingValue, maxValue)	The ceiling must be positive.
CombinedStat(startingValue)	
val max: ModularStat · var current: Double	<code>current</code> may go below zero.
val fractionOfMax: Double	
override fun applyMod / removeMod	Rescales <code>current</code> to preserve the fraction.
interface Capability	val id: String – stable and unique. §4.5
enum class CoreCapability	<code>MOVE("move")</code> , <code>ATTACK("attack")</code> .
open class Buff(name, durationTicks, statMods, suppressedCapabilities)	A timed effect. §4.6
var durationTicks: Int	Counted down each tick. Negative never expires.
val isExpired: Boolean	<code>durationTicks == 0</code> .
open fun onApplied / onTick / onExpired	Three hooks, all no-ops.
@Serializable wire types	Full field reference in §8.3.
GameObjectSnapshot(location, buffs)	<code>buffs</code> defaults to empty.
ColorSnapshot(r, g, b, a)	
BuffSnapshot(name, durationTicks, suppressedCapabilities)	Capability <i>ids</i> , not objects.

DECLARATION	NOTES
CombinedStatSnapshot(value, maxValue)	
sealed interface DrawableSnapshot	<code>id</code> , <code>subObjects</code> , <code>UNIDENTIFIED = 0L</code> , <code>infix from</code> .
VisibleObjectSnapshot @SerialName("visibleObject")	The three concrete kinds; the discriminator field is <code>type</code> .
ActorSnapshot @SerialName("actor")	
AliveSnapshot @SerialName("alive")	

9.3 · COM.SPARTANLABS.GAMING.EVENT

DECLARATION	NOTES
class EventBus	§6.2
fun subscribe(listener): Subscription	Ordered, strong reference.
fun publish(event)	Synchronous; re-entrant calls queue.
fun interface Listener	<code>fun onEvent(event)</code>
fun interface Subscription	<code>fun cancel()</code> , idempotent.
sealed interface GameEvent	§6.3
EntitySpawned(entity)	All are data classes.
EntityRemoved(entity)	
AttackIssued(attacker, target)	
AttackLanded(attacker, target, damage)	
DamageDealt(source?, target, amount)	
EntityDied(entity, killer?)	
AttackCancelled(attacker, formerTarget?)	
AttackEnded(attacker, formerTarget?, reason)	

9.4 · COM.SPARTANLABS.GAMING.SIMULATION

DECLARATION	NOTES
class SimulationLoop	§2.6
SimulationLoop(world, settings, onTick)	Does not start.
fun start(): Result<Unit>	Fails if running.
fun stop(): Result<Unit>	Joins up to 1 s. Idempotent.
val isRunning: Boolean	
fun advance(realElapsedNanos: Long)	Drive it yourself.
class LoopSettings	Both fields @Volatile.
var tickRateHz: Double = 20.0	Must be positive and finite.
var maxCatchUpTicks: Int = 5	Coerced to ≥ 1.
interface RandomSource	§2.7
nextDouble() · nextInt(n) · nextBoolean()	
class SeededRandom(seed: Long)	The default implementation.

9.5 · COM.SPARTANLABS.GAMING.SPATIAL

DECLARATION	NOTES
class Quadtree<N : Comparable<N>, E>	§5.2
fun insert(x, y, element)	Reuses an emptied slot on the path.
fun retrieveBox(minX, minY, maxX, maxY)	Half-open box; unspecified order.
fun remove(x, y, element)	Identity match; leaves a tombstone.
fun clear()	

9.6 · COM.SPARTANLABS.GAMING.NETWORKING

DECLARATION	NOTES
class GameServer : MultiConnectionUDPServer	§7
GameServer(maxConnections, onPlayerMessage, onPlayerInput)	Constructing starts the server.
val maxConnections: Int	
val playerCount: Int	
val playerNames: Set<String>	Detached copy.
fun broadcast(Iterable<VisibleObject>)	Filters invisible, snapshots, sends.
fun broadcast(List<DrawableSnapshot>)	Sends snapshots you took.
fun pushToAllPlayers(message)	Prefer over inherited <code>pushToAll</code> .
fun push(playerName, message)	Fails if not connected.
fun disconnect(playerName)	They may rejoin.
fun shutDown()	No restart afterwards.
const STATE_VERB · INPUT_VERB	"STATE" · "INPUT".
fun inputMessage(input): String	Builds the client's datagram.
override fun onClientConnect(connection)	Admission and routing setup.
data class MouseAction	§7.9
type · button · x · y	Window pixels, origin top-left.
enum class MouseActionType	MOVE , PRESS , RELEASE .

9.7A ·

COM.SPARTANLABS.GEOMETRY.SERIALIZATIONS

DECLARATION	NOTES
data class PointSnapshot(x, y)	Each with infix fun from . §8.7
data class DimensionsSnapshot(width, height)	
data class TwoDoublesSnapshot(first, second)	

9.7B · TOP-LEVEL OPERATORS

DECLARATION	NOTES
Double.plus(ModularStat)	Mirror operators so <code>3.0 + stat</code> resolves.
Double.minus(ModularStat)	
Double.times(ModularStat)	
Double.div(ModularStat)	
Double.compareTo(ModularStat)	
Double.plus / minus / times / div / compareTo (CombinedStat)	The same five against a <code>CombinedStat</code> 's <code>current</code> .

There is no `equals` operator – compare `.value` explicitly.

Recipes

Ten worked problems, each with the code and the reasoning behind the parts that are easy to get wrong.

10.1 A headless authoritative server

10.2 Spawning and ordering a unit

10.3 Haste, stun and damage over time

10.4 Firing both kinds of projectile

10.5 A kill feed from the event bus

10.6 Turning client input into orders

10.7 A deterministic regression test

10.8 A subclass with its own capability

10.9 Driving the timestep yourself

10.10 Debugging a divergence

QUICK REFERENCE

PROBLEM → CODE → THE PART THAT BITES

RECIPE 10.1

A headless authoritative server

The smallest complete server: a world, a loop, a socket, and a broadcast on every tick. Everything else in this chapter plugs into this skeleton.

```
import com.spartanlabs.gaming.gameobjects.*
import com.spartanlabs.gaming.networking.GameServer
import com.spartanlabs.gaming.simulation.LoopSettings
import com.spartanlabs.gaming.simulation.SimulationLoop
import java.util.concurrent.ConcurrentLinkedQueue

fun main() {
    val world = World() // seed is logged – note it down
    val pendingInput = ConcurrentLinkedQueue<Command>()

    val server = GameServer(
        maxConnections = 8,
        onPlayerMessage = { name, message -> // listener thread!
            pendingInput += Command.parse(name, message)
        },
        onPlayerInput = { name, action -> // listener thread!
            pendingInput += Command.fromMouse(name, action)
        })

    val loop = SimulationLoop(
        world = world,
        settings = LoopSettings(tickRateHz = 20.0),
        onTick = { // loop thread – safe
            while (true) (pendingInput.poll() ?: break).applyTo(world)
            server.broadcast(world.gameObjects.filterIsInstance<VisibleObject>())
                .onFailure { cause -> log.warn("broadcast failed", cause) }
        })

    loop.start().getOrNull()
    Runtime.getRuntime().addShutdownHook(Thread {
        loop.stop(); server.shutdown()
    })
}
```

The part that bites. The two callbacks run on WebTools' listener threads and the `onTick` callback runs on the loop thread. The queue is the only thing crossing between them, and it is drained *before* the tick's effects are broadcast so a command that arrives during a frame is applied at a defined boundary rather than mid-simulation. Draining after `world.tick()` — which is what `onTick` means — costs one frame of latency and buys determinism.

RECIPE 10.2

Spawning and ordering a unit

```
val player = Player("Spartak")

val hero = Alive(
    location    = Point(x = 100.0, y = 100.0),
    dimensions  = Dimensions(width = 40.0, height = 40.0),
    maxHealth   = 250.0
).apply {
    faction     = "blue"
    speed.base  = 6.0
    attackSpeed.base = 900.0           // ~19 ticks per swing, not 170
    deathResponse = Alive.DeathResponse.RESPAWN
    respawn     = Point(x = 50.0, y = 50.0)
    texture     = "hero.png"
    turns       = true
}

player.own(hero)
world.add(hero)           // numbers it, indexes it, sets hero.world

hero.destination = Point(x = 400.0, y = 250.0) // walk there and stop
hero.movement    = Movement.Persistent        // or: hold that spot
hero.issueAttack(enemy)                       // or: go and fight
```

The part that bites. The stock `attackSpeed` of `100.0` against an `attackTime` of `1.7` is one swing every 170 ticks. Raise it when you create the unit, or your first combat test will look broken (§4.7).

RECIPE 10.3

Haste, stun and damage over time

```
// +30 % move speed for 100 ticks. Name the mod after the source so two
// concurrent hastes do not remove each other (§4.2).
hero.applyBuff(Buff(
    name = "haste",
    durationTicks = 100,
    statMods = mapOf("speed" to StatMod("haste@boots-of-elvenkind", 0.30))))

// A flat bonus needs the type spelled out.
hero.applyBuff(Buff("blessing", 200,
    statMods = mapOf("damage" to
        StatMod("blessing", 15.0, StatMod.Type.ADDITIVE))))

// A stun is both core capabilities held down. No stat mods at all.
target.applyBuff(Buff(
    name = "stun",
    durationTicks = 40,
    suppressedCapabilities = setOf(CoreCapability.MOVE, CoreCapability.ATTACK)))

// Damage over time: subclass and use the per-tick hook.
class Poison(private val perTick: Double, ticks: Int) : Buff("poison", ticks) {
    override fun onTick(target: GameObject) {
        (target as? Alive)?.health?.let { it.current -= perTick }
    }
}
target.applyBuff(Poison(perTick = 2.5, ticks = 60))

// Refresh-instead-of-stack, at the point of application.
target.dispel("poison")
target.applyBuff(Poison(perTick = 2.5, ticks = 60))
```

The part that bites. `Poison` writes to `health.current` directly, so it publishes no `DamageDealt` and sets no killer. A poison kill will show up as an `EntityDied` with a `null` killer unless you publish something yourself (§4.8).

RECIPE 10.4

Firing both kinds of projectile

```
// A guided missile. Hand it the world's quadtree so its broad phase
// sees the same index everything else does.
val missile = HomingProjectile(
    location    = Point(caster.location),    // a copy, not the caster's own Point
    dimensions  = Dimensions(width = 12.0, height = 12.0),
    damage      = 45.0,
    target      = victim,
    quadtree    = world.quadtree
).apply { speed.base = 9.0; texture = "missile.png"; turns = true }
world.add(missile)

// A piercing beam along a heading, alive for 30 ticks.
val beam = DirectionalProjectile(
    location      = Point(caster.location),
    dimensions    = Dimensions(width = 20.0, height = 6.0),
    damage        = 20.0,
    directionAngle = caster.angle,
    maxDuration   = 30,
    quadtree      = world.quadtree
).apply { speed.base = 14.0 }
world.add(beam)

// Nothing removes a spent projectile. Sweep them once per tick.
world.gameObjects
    .filterIsInstance<Projectile>()
    .filterNot { it.active }
    .forEach { world.removeList += it }
```

The part that bites. Two things. Pass a *copy* of the caster's position — `Point(caster.location)` — or the projectile and the caster will share one mutable point and move together. And keep the projectile's `speed` below roughly the summed half-extents of it and its target, or it will step over the target between ticks and never register a hit (§3.7).

RECIPE 10.5

A kill feed from the event bus

```
class Scoreboard(world: World) {
    private val kills = mutableMapOf<String, Int>()
    private val feed = mutableListOf<String>()

    private val subscription = world.events.subscribe { event ->
        when (event) {
            is GameEvent.EntityDied -> {
                val victim = event.entity.owner?.name ?: "a neutral"
                val killer = event.killer?.owner?.name
                if (killer != null) kills.merge(killer, 1, Int::plus)
                feed += killer?.let { "$it killed $victim" } ?: "$victim died"
            }
            is GameEvent.DamageDealt ->
                damageMeter.add(event.source?.entityId, event.amount)
            else -> {}
        }
    }

    fun close() = subscription.cancel()    // or it lives as long as the world
}
```

The part that bites. `killer` is `null` for every death that did not come from a swing — projectiles, buffs, direct health edits — so the feed needs the environmental branch. And the listener runs inside the tick: append to a list, do not render or write to disk here.

RECIPE 10.6

Turning client input into orders

```
// A command carries only ids and numbers, so it is safe to hand across threads.
sealed interface Command {
    val player: String
    fun applyTo(world: World)

    data class MoveTo(override val player: String,
                     val unit: EntityId, val x: Double, val y: Double) : Command {
        override fun applyTo(world: World) {
            val actor = world.byId(unit) as? Actor ?: return // stale id → drop
            if ((actor as? Alive)?.owner?.name != player) return // not yours → drop
            actor.cancelIfAttacking()
            actor.destination = Point(x = x, y = y)
        }
    }
}

// On the listener thread: translate window pixels into world space and queue.
onPlayerInput = { name, action ->
    if (action.type == MouseActionType.PRESS && action.button == 1) {
        val camera = cameras[name] ?: return@GameServer
        pendingInput += Command.MoveTo(
            player = name,
            unit    = selections[name] ?: return@GameServer,
            x       = camera.originX + action.x / camera.scale,
            y       = camera.originY + action.y / camera.scale)
    }
}
```

The part that bites. Three separate traps. `MouseAction` carries *window pixels*, so the server needs the client's camera to convert them (§7.9). The command must name its target by `EntityId`, because a list index means nothing a frame later — and `byId` returning `null` is exactly how you learn the target has died. And the ownership check belongs on the server: a client can send any id it likes.

RECIPE 10.7

A deterministic regression test

```
class SeededCombatTest {
    private fun runBattle(seed: Long): List<String> {
        val world = World(seed)
        val log = mutableListOf<String>()
        world.events.subscribe { log += "${world.tickCount}: $it" }

        val a = Alive(Point(0.0, 0.0), Dimensions(10.0, 10.0), 100.0)
        val b = Alive(Point(30.0, 0.0), Dimensions(10.0, 10.0), 100.0)
        b.evasion.base = 0.25
        listOf(a, b).forEach { it.attackSpeed.base = 2000.0; world.add(it) }

        a.issueAttack(b)
        repeat(200) { world.tick() }
        return log
    }

    @Test fun `the same seed produces the same battle`() {
        assertEquals(runBattle(seed = 1234L), runBattle(seed = 1234L))
    }

    @Test fun `a different seed produces a different battle`() {
        assertNotEquals(runBattle(1234L), runBattle(9999L))
    }
}
```

The part that bites. Determinism holds only if *everything* is identical: the seed, the order of `add` calls, and the tick on which each external call is made. Add the objects in a fixed order — `listOf(a, b).forEach`, never a set — and make your assertions about the event log rather than about wall-clock behaviour.

RECIPE 10.8

A subclass with its own capability

```
enum class MageCapability(override val id: String) : Capability {
    CAST("cast")    // the id crosses the wire in a BuffSnapshot – keep it stable
}

class Caster(location: Point, dimensions: Dimensions, maxHealth: Double)
    : Alive(location, dimensions, maxHealth) {

    val mana = CombinedStat(startingValue = 100.0, maxValue = 100.0)
    var manaRegen = ModularStat(base = 0.5)

    // Getters, not stored properties, so a reassigned stat stays reachable.
    override val capabilities = super.capabilities + MageCapability.CAST
    override val stats get() = super.stats +
        mapOf("mana" to mana, "manaRegen" to manaRegen)

    override fun onUpdate() {
        super.onUpdate()                // always first
        if (!can(MageCapability.CAST)) return // silenced
        mana.current = (mana + manaRegen).coerceAtMost(mana.max.value)
    }
}

// A silence: the same mechanism the core layers use for root and disarm.
mage.applyBuff(Buff("silence", durationTicks = 60,
    suppressedCapabilities = setOf(MageCapability.CAST)))
```

The part that bites. `stats` must be a getter (`override val stats get() = ...`); declaring it as a stored property captures the stat objects at construction and quietly stops tracking any you later reassign. And `super.onUpdate()` first, always — omit it and the caster stops moving, fighting and dying.

RECIPE 10.9

Driving the timestep yourself

```
// You already have a loop – a game engine frame, a test harness, a
// scheduler. Take the accumulator without the thread.
val loop = SimulationLoop(world, LoopSettings(tickRateHz = 30.0))
var last = System.nanoTime()

while (running) {
    val now = System.nanoTime()
    loop.advance(now - last)    // runs 0..maxCatchUpTicks ticks
    last = now
    render()
}

// Or skip the accumulator entirely and step by hand – a test does this.
repeat(100) { world.tick() }
```

The part that bites. Never call `advance` while `start()`'s thread is running, or concurrently with itself — the accumulator is not synchronised. And remember that time owed beyond `maxCatchUpTicks` is discarded, so after a long stall the simulation is permanently that much behind wall-clock time. That is the intended behaviour, not a bug to compensate for.

RECIPE 10.10

Debugging a divergence

Two runs that should match do not — a replay differs from the original, or two servers disagree. The event log is the bisection tool.

```
// 1. Pin the seed. It was written to the log at INFO on construction:
//      "World created with seed -4820193857..."
val world = World(seed = -4820193857L)

// 2. Record a canonical trace of both runs.
val trace = mutableListOf<String>()
world.events.subscribe { event -> trace += "%06d %s".format(world.tickCount, event) }

// 3. Add a per-tick position digest – events alone miss silent drift.
fun digest() = world.gameObjects.joinToString("|") {
    "${it.entityId.raw}:${it.location.x},${it.location.y}"
}

// 4. Diff the two traces. The first differing line names the tick.
```

IF THE DIVERGENCE IS...	LOOK FOR
an evasion or other roll	A call to <code>Math.random()</code> or <code>Random.Default</code> in your own code instead of <code>world.rng</code> .
the <i>order</i> of two events on the same tick	Objects added in a different order, or iteration over a <code>HashSet</code> / <code>HashMap</code> .
a position, one tick early or late	Input applied on arrival instead of on a tick boundary (§10.6).
everything, from an arbitrary tick	A second thread touching the world — a callback, a UI, a broadcast built off-thread.
only under load	The catch-up cap discarding time (§2.6): the two runs did different numbers of ticks.

Appendices

A–E

Every default in one table, the vocabulary, a symptom-first troubleshooting index, the build commands, and the parts you cut out.

A Every default in the library

B Glossary

C Troubleshooting

D Build, test and release tasks

E Printable dividers and spine label

A Every default in the library

TYPE	MEMBER	DEFAULT	NOTE
GameObject	location	Point() – (0, 0)	A mutable point in a stable cell.
	active	true	
	entityId	EntityId.UNASSIGNED (0)	Until a world numbers it.
	capabilities / stats	empty	Extended by each layer.
VisibleObject	area.dimensions	25.0 × 25.0	
	color	Color(255,255,255,255)	Opaque white.
	texture	"default.png"	
	angle	0	Degrees CCW from +x, normalised to 0–359.
	turns	false	Renderers ignore <code>angle</code> unless true.
	visible	true	Tracks <code>active</code> .
Actor	speed.base	10.0 units / tick	200 units/s at 20 Hz.
	movement	Movement.Targeting	
	destination	Point(location)	A copy of the start position.
Alive	damage.base	10.0	
	attackTime.base	1.7	Progress threshold for a swing.
	attackSpeed.base	100.0	Progress accrues at <code>value / 10000</code> per tick.
	attackRange.base	750.0 units	Very long. Usually wants lowering.
	evasion.base	0.0	Probability in 0..1. Never dodges.
	faction	"neutral"	<code>Alive.DEFAULT_FACTION</code> . Never read by the library.
	deathResponse	DeathResponse.REMOVAL	
	respawn	Point(location)	The creation position.
	world / owner	null	<code>World.add</code> sets the first.
	health bar height	20 % of actor height	Colour <code>Color(255, 0, 0)</code> .
StatMod	type	Type.MULTIPLICATIVE	
	stackingType	StackingType.ADDITIVE	So <code>StatMod("x", 0.25)</code> is "+25 %".
Buff	statMods / suppressedCapabilities	empty	A negative <code>durationTicks</code> never expires.
World	seed	Random.nextLong()	Logged at INFO on construction.
	tickCount	0	
LoopSettings	tickRateHz	20.0	Must be positive and finite.
	maxCatchUpTicks	5	Coerced to ≥ 1.
SimulationLoop	thread name	"gametools-sim-loop"	Daemon. <code>stop()</code> joins for 1000 ms.

TYPE	MEMBER	DEFAULT	NOTE
	onTick	{ }	
DirectionalProjectile	searchRadius	dimensions.width × 3	maxDuration must be positive.
GameServer	onPlayerMessage / onPlayerInput	no-ops	Verbs "STATE" and "INPUT" ; keepalive KA ≈ 20 s.
DrawableSnapshot	UNIDENTIFIED	0L	Also the decode default for id.
GameObjectSnapshot	buffs	emptyList()	Omitted from JSON when empty.

B Glossary

Actor	A visible object that moves itself.	Quadtree	The point-region spatial index, rebuilt each tick.
Alive	An actor with health and combat.	Root	A buff suppressing MOVE.
Broad phase	A cheap, inclusive proximity query — the quadtree.	Seed	The number a world's RNG is built from.
Buff	A timed bundle of stat mods and capability suppressions.	Settled	A Targeting actor that has arrived and stopped.
Capability	A named ability a buff can hold down.	Snapshot	An immutable, serializable copy taken at a known instant.
Catch-up cap	The most ticks one advance will run.	Spiral of death	Catch-up ticks costing more time than they replay. Guarded by the cap.
Determinism	Same seed + same inputs ⇒ same result.	Stat	A ModularStat or CombinedStat.
Disarm	A buff suppressing ATTACK.	Stun	A buff suppressing both core capabilities.
EntityId	A stable, per-world, never-reused object handle.	Sub-object	A visible child drawn relative to its parent, not ticked independently.
Faction	A free-text side label the library never reads.	Tick	One call to World.tick(); the engine's unit of time.
Fixed timestep	A simulation step of constant size, regardless of real frame length.	Tombstone	A quadtree node whose element was removed, left as a reusable slot.
Hook	A protected open fun on...() a subclass overrides.	Tunnelling	A fast object stepping past a target between ticks without overlapping it.
Mod	A named adjustment applied to a stat.	Unit	One unit of world space. The library fixes no scale.
Narrow phase	The exact overlap test on a broad-phase candidate.	Verb	The first token of a datagram — INPUT, STATE, KA.

C Troubleshooting

SYMPTOM	ALMOST ALWAYS	FIX
No log output at all.	No slf4j binding on the classpath.	Add <code>logback-classic</code> as a runtime dependency. §1.4
A dead unit stays in the world at zero health, and the log says “died with REMOVAL but has no world”.	It was added with <code>gameObjects +=</code> , so <code>Alive.world</code> is null.	Use <code>world.add(unit)</code> . §2.2
<code>byId</code> returns null for something you just added.	Same cause — direct list addition is only indexed from the next tick.	Use <code>add()</code> , or tick once first. §2.4
Two units in range never seem to hit each other.	The stock swing is 170 ticks — 8.5 s at 20 Hz.	Raise <code>attackSpeed.base</code> or lower <code>attackTime.base</code> . §4.7
A buff applies but the number does not change.	A <code>MULTIPLICATIVE</code> mod with <code>SETTING</code> or <code>INCREMENTING</code> stacking contributes nothing.	Use <code>ADDITIVE</code> type for those stacking modes. §4.2
A buff applies but the log warns about an unknown stat.	The key is not in that object's <code>stats</code> map.	Check the spelling, or expose the stat. §4.4
One buff expires and two effects disappear.	Both buffs used the same <code>StatMod</code> name on the same stat.	Namespace mod names per source. §4.2
A projectile flies through its target.	Tunnelling: <code>speed</code> exceeds the combined size.	Slow it, enlarge it, or tick faster. §3.7
An object exactly on the edge of a query window is missed.	The query box is half-open — low edges are excluded.	Widen the range slightly. §5.3
Memory and tick cost grow steadily over a long match.	Spent projectiles are deactivated but never removed.	Sweep inactive projectiles into <code>removeList</code> . §10.4
A kill feed shows “killed by nobody”.	Projectile and buff damage record no attacker.	Attribute it yourself. §4.8
One player stops receiving updates after ~30–60 s of standing still.	Their client is not sending <code>KA</code> keepalives, and the NAT mapping expired.	Send <code>KA</code> every ~20 s from the handshake socket. §7.7
A second <code>GameServer</code> fails to bind.	The common port is fixed; one instance per JVM.	<code>shutDown()</code> the first. §7.8
State corrupts, or crashes appear, only with several players.	The world is being mutated from a listener thread.	Queue commands; drain on the tick thread. §10.6
A client's <code>INPUT</code> seems ignored.	Malformed JSON is logged at <code>warn</code> and dropped.	Build it with <code>GameServer.inputMessage</code> . §7.4
A snapshot has no <code>id</code> field.	JSON omits values equal to their default, and an unowned object's id is <code>0</code> .	Treat it as optional, defaulting to 0. §8.6
The simulation is permanently behind wall-clock time after a pause.	The catch-up cap discarded the owed time.	Intended. Raise <code>maxCatchUpTicks</code> only if you understand the risk. §2.6
A listener threw and nothing happened.	The bus catches and logs at <code>warn</code> .	Read the log; the tick is never aborted. §6.2
The log fills with angle-normalisation warnings.	An angle is being accumulated without wrapping.	Wrap with <code>mod 360</code> before assigning. §3.2
A resized unit's health bar is the wrong width.	The bar's full width is captured from the actor's size at construction.	Rebuild the bar, or size units at construction. §3.5

D Build, test and release tasks

GRADLE TASKS

COMMAND	DOES
<code>./gradlew build</code>	Compile and run every test level.
<code>./gradlew test</code>	Every level, without the rest of <code>build</code> .
<code>./gradlew componentTest</code>	Level 2 — fast, isolated.
<code>./gradlew integrationTest</code>	Level 3 — binds real UDP ports.
<code>./gradlew deterministicTest</code>	Level 4a — pure logic and laws.
<code>./gradlew e2eTest</code>	Level 4b — client↔ server round trip.
<code>./gradlew nonfunctionalTest</code>	Level 4c — scalability and robustness.
<code>./gradlew dokkaGeneratePublicationHtml</code>	The API docs, aggregated across modules.

WHY THE TEST LEVELS ARE SEPARATE TASKS

So CI can gate them independently, and so the three levels that bind a UDP port can declare a single-permit Gradle build service. Gradle runs independent tasks in parallel, including across modules, and two `GameServer`s cannot share the fixed common port — the shared service is what serialises them.

TEST HIERARCHY

LEVEL	PACKAGE	SCOPE
2	<code>...testing.component</code>	Isolated component and business logic.
3	<code>...testing.integration</code>	External interfaces. Binds ports.
4a	<code>...testing.deterministic</code>	Input/output and law-based tests.
4b	<code>...testing.e2e</code>	Full round trip. Binds ports.
4c	<code>...testing.nonfunctional</code>	Scalability and robustness. Binds ports.

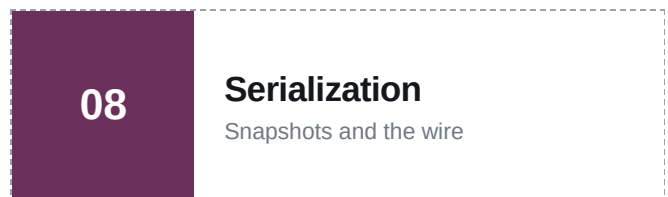
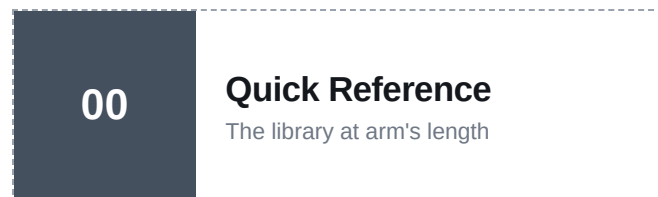
One test class per file, by house rule. A level with no tests in a module is a green no-op.

TOOLCHAIN AND PUBLISHING

ITEM	VALUE
Language	Kotlin 2.2.0 (JVM)
Toolchain	JVM 23
Test framework	JUnit 5 · kotlin-test
Docs	Dokka 2.0.0 (V2 plugin mode)
Publishing	Vanniktech → Maven Central, all publications signed
Version scheme	Major.Feature.MinorChange, trailing letter = bug-fix
Licence	Apache 2.0

E Printable dividers and spine label

Print this page on card stock, cut along the dashed lines, and place each divider immediately before its chapter opener. The coloured lug matches the swatch beside that chapter in the contents. Trim the spine label to your binder's sleeve.



01	Architecture Layers, seams, idioms, modules	09	API Reference Every declaration
02	World & Simulation The tick and the timestep	10	Recipes Ten worked problems
03	Game Objects The entity chain	A–E	Appendices Defaults, glossary, fixes
04	Stats & Combat Numbers, buffs, swings	GAMETOOLS REFERENCE FOLDER · v3.1.0	
05	Spatial The quadtree	GAMETOOLS REFERENCE FOLDER · v3.1.0	
06	Events The typed bus	KEEPING THIS FOLDER CURRENT The source is docs/reference/gametools-reference.html. Rebuild the PDF with node docs/reference/build-reference-pdf.mjs, which re-measures the contents' page numbers and re-renders. Reprint whichever chapters changed — every chapter starts on a fresh page, so a chapter can be swapped without disturbing the rest.	
07	Network UDP, protocol, threading		

GameTools 3.1.0 · Apache License 2.0 · Spartan Laboratories. Written against **master** as of 6 September 2026. Where this folder and the source disagree, the source is right — and the disagreement is a bug in one of them.